

Trading Engine

Code-Level Execution Manual

How one Bitstamp message becomes reconstructed order-book state — traced through the code that exists today.

Repository	TradingEngineProject (local main)
Commit	d00e244
Verified	2026-09-15, against a clean out-of-tree configure + build
Test suite	304 tests registered — 303 pass, 1 fails (see §2)
Source size	~10,900 lines C++ across 118 tracked files
Method	Every behavioural claim below was read from the implementation, not from the plan documents.

READ THIS FIRST

This manual documents **what the code does today**. Where a project document claims something the code does not support, the disagreement is called out explicitly rather than smoothed over. Three such disagreements were found, plus one dead-code defect and one data-loss condition. They are collected in **§2 Current system status** and flagged again at the point where each one matters.

WHAT DOES NOT EXIST YET

There is **no** engine loop, strategy, portfolio, execution venue, risk gate or decision gate. Those headers are empty placeholders. Everything this document traces end-to-end stops at *reconstructed book state*. §27 maps exactly where the implemented system ends and the designed-but-unbuilt system begins.

§ Contents

Thirty chapters, grouped into six parts.

I · ORIENTATION

- 1 Executive summary
- 2 Current system status & findings
- 3 Architecture overview
- 4 Domain terminology

II · INGESTION

- 5 Data sources
- 6 Capture & persistence
- 7 Raw Bitstamp messages
- 8 Parsing & decoding
- 9 Exact-integer conversion
- 10 The normalized event model

III · ORDERING

- 11 Snapshot & bootstrap
- 12 Classification
- 13 Two streams, one sequence
- 14 MergeCursor

IV · THE BOOK

- 15 OrderBook internal architecture
- 16 ADD lifecycle
- 17 MODIFY lifecycle
- 18 REMOVE lifecycle
- 19 Order state machine

V · TRADES & THE LOOP

- 20 Trades are not order events
- 21 TradeReconciler in depth
- 22 The full replay loop
- 23 Errors & defensive checks
- 24 Determinism

VI · EVIDENCE & REFERENCE

- 25 Tests as evidence
- 26 One full worked scenario
- 27 Implemented vs. designed
- 28 The decision frontier (live work)
- 29 Code-to-runtime cross reference
- 30 Glossary & one-page cheat sheet

Visual language

Colour is used consistently and is always paired with a text label, so the document survives greyscale printing.

-  raw / incoming
-  parse / transform
-  order / reconcile
-  valid mutation
-  error / rejection
-  persistence

Status vocabulary

 **IMPLEMENTED** real code, real tests, exercised today  **PARTIAL** exists but incomplete  **PLACEHOLDER** empty file reserving a name  **DESIGNED** decided on paper, not built  **BROKEN** present but failing

1 Executive summary

What this system is, in one page.

This repository is a **deterministic L3 market-data replay engine**. Its single working capability today is this: take a recorded Bitstamp order-by-order feed plus an independently fetched book snapshot, and reconstruct — byte-for-byte reproducibly — what the order book looked like at every microsecond in between.

"L3" is the distinguishing word. An L2 feed tells you *50 BTC rests at \$100,000*. An L3 feed tells you *which individual orders* make up that 50 BTC, each with its own ID and its own position in the queue. Bitstamp's `live_orders` channel is one of the few freely available L3 crypto feeds, which is why it is the primary source here. That granularity is what eventually makes queue-position modelling possible — knowing not just that you can trade at a price, but how many orders sit ahead of yours.

The pipeline in one sentence

A raw JSON frame is decoded into a fixed-size venue-neutral `OrderEvent`, merged against a second stream of trades under one centrally-owned tie-break rule, filtered by a stateful classifier, applied to a reference `OrderBook` that maintains both price-ordered levels and an ID index, corrected where the trade stream disagrees with the order stream, and fingerprinted so two runs can be proven identical.

What makes it non-trivial

Problem	How this codebase answers it
Money cannot use floats	Every price and quantity is an <code>int64</code> in exact venue ticks. Text→integer conversion is hand-written with pre-multiplication overflow guards. A C1 architecture guard fails the build if a <code>double</code> appears in financial code.
Two streams, one truth	Orders and trades arrive as independent time-ordered streams. <code>MergeCursor</code> is the <i>only</i> place allowed to decide their interleaving, so the policy cannot drift between call sites.
The venue reports fills twice	Bitstamp's <code>live_orders</code> already subtracts a fill from the resting quantity, <i>and</i> the trade stream reports the same fill. Naïvely applying both deletes live orders. <code>TradeReconciler</code> corrects only the uncovered shortfall, keyed by <code>(orderId, timestamp)</code> .
Reproducibility must be provable	<code>OrderBook::digest()</code> and a running applied-event digest hash <i>semantic fields in canonical order</i> — never memory layout — so two books holding identical liquidity agree across compilers, allocators and insertion orders.
Silent data loss must be impossible	Replay counts every input into exactly one of <code>beforeSeed</code> , <code>read</code> , or <code>afterCutoff</code> . The three must sum to the input size, so an event skipped by a merge bug leaves an arithmetic trace.

Maturity, honestly stated

The ingestion-to-book half of the system is real, tested and exercised against a 29,000-event real capture. The trading half — strategy, portfolio, venue, risk, the engine loop itself — is **not written**. Its headers are one-line comments reserving a filename. This document spends most of its pages on the half that exists, and is explicit in §27–§28 about the half that does not.

INDUSTRY FRAMING

The architecture here is a **capture/replay (record-and-replay) system**, the same shape used by exchange tick-capture infrastructure and by deterministic simulation frameworks generally — FoundationDB's simulation testing and Antithesis are the well-known modern examples. The governing idea is identical: if execution is a pure function of an ordered input log, then a bug seen once can be reproduced exactly, forever. Everything unusual about this codebase — the centralized tie-break, the semantic digests, the event accounting — exists to protect that property.

2 Current system status & findings

Verified by clean out-of-tree configure, build and full CTest run on 2026-09-15.

2.1 Build and test reality

The project's own guidance warns that `cmake --build build` can report success against stale objects. That warning turned out to be an understatement: the checked-in `build/` directory does not configure at all.

```
$ cmake --build build -j
CMake Error at /opt/homebrew/share/cmake/Modules/FindThreads.cmake:129 (message):
  FindThreads only works if either C or CXX language is enabled
Call Stack (most recent call first):
  cmake/dependencies.cmake:10 (find_package)
  CMakeLists.txt:31 (include)
-- Configuring incomplete, errors occurred!
```

This is **not** a source defect. `CMakeLists.txt:4` declares `project(TradingEngine VERSION 0.1.0 LANGUAGES CXX)` correctly. A fresh out-of-tree configure of the same source succeeds immediately and completely. The fault is entirely in the stale `build/` cache.

Measurement	Reported in <code>status.md</code>	Actually observed
In-tree <code>build/</code> usable	implied yes	No — configure fails
Tests registered	303	304
Tests passing	303 of 303	303 of 304 — one fails
Failure is reproducible	—	Yes: 3 of 3 consecutive full runs

FINDING 1 — THE HEADLINE EVIDENCE TEST IS FAILING

`BitstampJoinedCapture.RealCaptureReplaysToCheckpointWithNoResiduals` fails. This is the test that both `status.md` and ADR 0013 cite as proof of the project's strongest correctness claim: *"0 of 4,533 level residuals"* against a real 29,404-event capture. It is not passing on this machine, and it is not passing for a subtle reason.

2.2 Why that test fails: the corpus is hollow

The failure output shows every count reading zero, rather than reading a wrong value:

```
tests/unit/test_bitstamp_joined_capture.cpp:363: Failure
  joinedCapture.jc_captureOrderEvents.size() Which is: 0
  29404U Which is: 29404
tests/unit/test_bitstamp_joined_capture.cpp:364: Failure
  joinedCapture.jc_tradeEvents.size() Which is: 0
  84U Which is: 84
```

The capture loaded successfully and produced nothing. Inspecting the fixture on disk explains it — the files have a size in their metadata but **zero allocated blocks**:

```
$ stat -f "blocks=%b size=%z %N" data/raw/bitstamp-btcusd-20260822T000512Z/*
blocks=560 size=285140 checkpoint-0000.snapshot <-- real data
blocks=8 size=2680 manifest.json <-- real data
blocks=0 size=7494082 segment-0000.frames.jsonl <-- HOLLOW
blocks=0 size=14684968 segment-0000.jsonl <-- HOLLOW
blocks=560 size=284735 segment-0000.snapshot <-- real data

$ wc -l segment-0000.jsonl
0 <-- 14 MB, zero lines
$ head -c 200 segment-0000.jsonl
<-- returns nothing
```

The project root lives under `~/Desktop`, which is an iCloud-synced location. These are evicted placeholder files: macOS retains the size in metadata while the content is gone. Reading them does not fault and does not trigger a successful restore — it silently returns empty. The condition is widespread across the corpus:

Evicted (blocks = 0)	Declared size
<code>data/raw0ld/btcusd-live-orders.jsonl</code>	126,085,157
<code>data/raw/...20260827T235604Z/segment-0000.frames.jsonl</code>	34,325,370
<code>data/raw/...20260822T000512Z/segment-0000.jsonl</code>	14,684,968
<code>data/raw/...20260822T000512Z/segment-0000.frames.jsonl</code>	7,494,082
<code>data/raw/...20260828T003754Z/segment-0000.jsonl</code>	2,732,384
...and roughly a dozen more, including several <code>manifest.json</code> files.	

FINDING 2 — THE SKIP-GUARD IS DEFEATED BY PARTIAL EVICTION

The test is *designed* to skip when corpus data is absent:

```
if (!std::filesystem::exists(capture / "manifest.json")) {
    GTEST_SKIP() << "OPTIONAL EVIDENCE NOT RUN: ...";
}
```

The guard checks **only** `manifest.json`. That file survived eviction; the payload it points at did not. So the guard passes, the test runs, and it reports a hard failure where the author's intent was a clean skip. A guard that checked the payload is non-empty — not merely that the manifest exists — would restore the intended behaviour.

Source: `tests/unit/test_bitstamp_joined_capture.cpp:349–354`

CONSEQUENCE FOR THIS DOCUMENT

ADR 0013's measured tables (637 of 637 fills covered; 0 uncovered; 0 of 4,533 residuals) are reported throughout this manual **as historical measurements recorded in the ADR**, because they cannot currently be re-derived on this machine. They are not re-verified claims. Wherever such a number appears, it is marked as *ADR-recorded*.

2.3 A dead-code defect in the order book

`OrderBook::applyAdd()` contains unreachable duplicated code after its `return` statement:

`src/book/order_book.cpp:86–90` — `applyAdd()`, final lines

```
return Result<ApplyOutcome, ApplyError>::success(
    ApplyOutcome{.createdLevel = createdLevel});
    orderIndex_.emplace(orderEvent.order_id, orderLocator); // <-- unreachable
}
```

Behaviour is **correct** today: the real insertion happens earlier at line 74, inside the `try` block that handles `std::bad_alloc`. The stray line is a duplicate that can never execute. It is harmless but actively misleading — it reads as though insertion happens after the success value is constructed, which is the opposite of the rollback ordering the function is carefully built around. Compilers do not warn here by default because the statement is well-formed.

FINDING 3 — REMOVE, DON'T RATIONALISE

Deleting line 88 is a no-op behaviourally and removes a genuine trap for the next reader. The surrounding rollback logic (§16) is subtle enough without a phantom second insertion in it.

2.4 Documentation drift

Document	Claim	Status against code
status.md	"passes 303 of 303 tests"	304 registered; 303 pass, 1 fails
status.md	"the principal end-to-end correctness path does not disappear on a fresh checkout"	Half true. The <i>synthetic</i> committed fixture does pass and does protect the wiring. But the real-corpus test does not skip cleanly when data is missing — it fails (Finding 2).
status.md	Engine/strategy/venue/portfolio are placeholders	Accurate. Verified: all are one-comment files.
status.md	<code>isTrusted()</code> / <code>marketShape()</code> split is done	Accurate. Both exist as described.
TOD0.md	Decode merge closed; one <code>decodeCapturedOrder</code> per frame	Accurate. <code>segment_loader.cpp:133</code> makes exactly one call.

2.5 Uncommitted work in the tree

At the time of writing, `git status` shows only untracked learning-toolkit directories (`.claude/commands/`, `.claude/skills/`). All source modifications discussed in earlier handoffs are committed as of `d00e244`.

2.6 Summary scorecard

Subsystem	Status	Evidence
Text→integer conversion	IMPLEMENTED	238-line test file, overflow cases covered
Order/trade/snapshot decoding	IMPLEMENTED	Unit tests + real-line fixtures
Manifest & segment loading	IMPLEMENTED	275-line manifest test, 12 error variants
MergeCursor ordering	IMPLEMENTED	195-line test; ADR 0013 accepted
OrderBook (add/modify/remove)	IMPLEMENTED	385-line test + structural oracle
TradeReconciler	PARTIAL	Unit-tested; correction path never fired on real data (ADR-recorded: 0 of 753 trades)
Replay controller	IMPLEMENTED	370-line test; real-corpus gate currently failing (§2.2)
BookHealth	PARTIAL	State machine works; nothing calls it — no production caller exists
Telemetry / v3 tape	PARTIAL	Format + reader/writer exist; replay does not consume the tape
Engine loop	PLACEHOLDER	<code>engine.hpp</code> is 5 lines of comment
Strategy / NoopStrategy	PLACEHOLDER	<code>strategy.hpp</code> , <code>noop_strategy.cpp</code> empty
Portfolio / accounting	PLACEHOLDER	<code>portfolio.hpp</code> empty; test file empty too
ExecutionVenue / risk	PLACEHOLDER	<code>venue.hpp</code> , <code>risk.hpp</code> empty
DecisionGate	DESIGNED	ADR 0014 <i>proposed</i> , 8 open questions (§28)

3 Architecture overview

The complete implemented path, with the file that owns each stage.

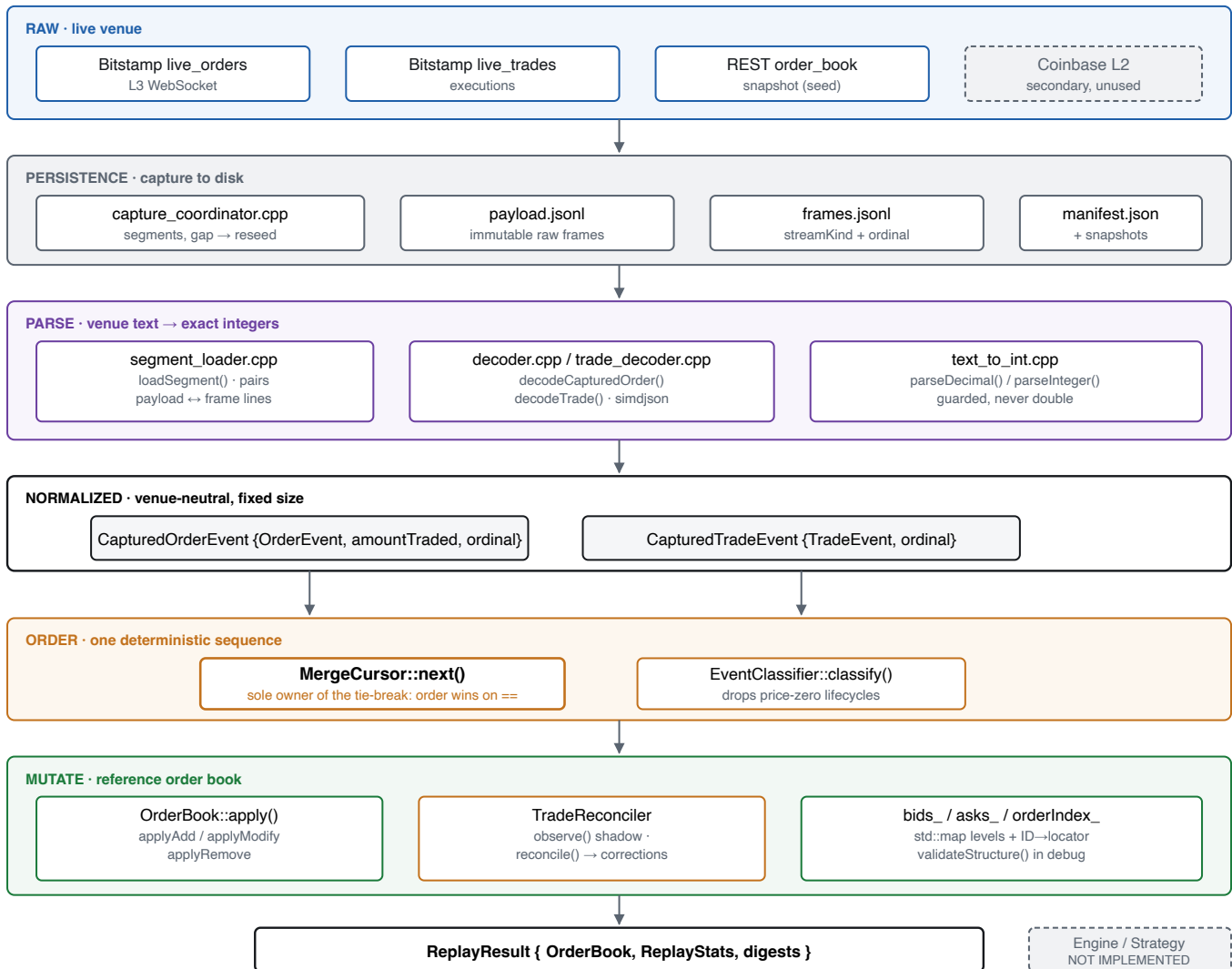


Figure 3.1 — The implemented pipeline. Dashed grey boxes do not exist in code. Every solid box names the file that owns that stage.

3.1 Directory map

Path	Status	Responsibility
src/core/	IMPL	Exact text→integer conversion, time, venue/instrument identity
src/feed/bitstamp/	IMPL	Venue-specific decode, snapshot parse, bootstrap, classifier, replay controller
src/feed/	IMPL	Venue-neutral: MergeCursor, TradeReconciler
src/book/	IMPL	OrderBook, PriceLevel, BookHealth
src/capture/	IMPL	Manifest reading, segment loading, capture coordination
src/telemetry/	PARTIAL	v3 binary tape format, segment reader/writer, legacy v2 recorder
src/util/	IMPL	Byte buffer, intrusive list, memory pool, SPSC queue
include/te/engine/	PLACEHOLDER	4 empty headers: engine, portfolio, risk, strategy
include/te/venue/	PLACEHOLDER	5 empty headers: venue, simulated, latency, queue, coinbase
strategies/	PLACEHOLDER	noop, imbalance, market_maker — all empty

3.2 The dependency rule that shapes the layout

Venue-specific code lives under `feed/bitstamp/`; anything that could serve a second venue lives one level up in `feed/`. `MergeCursor` and `TradeReconciler` are deliberately in the neutral layer and know nothing about Bitstamp. The normalization boundary is the decoder: after `decodeCapturedOrder()` returns, no downstream component can tell which venue produced the event.

INDUSTRY FRAMING

This is the **anti-corruption layer** from domain-driven design, and it is standard in market connectivity: FIX engines normalize dozens of dialect quirks into one internal order model at the session boundary, precisely so the matching/risk layers never branch on counterparty. The cost is a translation step per venue; the benefit is that `OrderBook` has no venue conditionals anywhere in its 279 lines.

4 Domain terminology

Three concepts this project deliberately refuses to merge.

The repository keeps a short glossary in `CONTEXT.md` whose entire purpose is to stop three distinct ideas collapsing into one boolean. That collapse is the most common design error in market-data systems, so the separation is worth stating precisely.

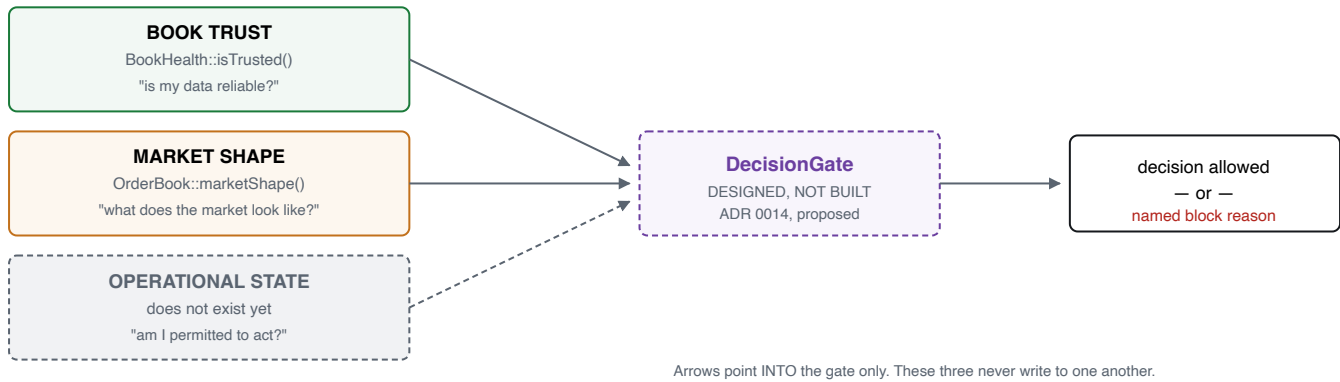


Figure 4.1 — The three-input model from ADR 0014. Dashed = not implemented.

Term	Means exactly	Does NOT mean	Where it lives
Book trust	The reconstruction rests on a valid snapshot and an uninterrupted, successfully processed event stream.	That the market is tradeable. A perfectly trusted book can be empty.	book_health.hpp isTrusted()
Market shape	The visible relationship between best bid and best ask: empty, one_sided, locked, crossed, open.	That the data is reliable. A crossed book may be perfectly-received truth.	order_book.cpp marketShape()
Decision readiness	The combination of trust, shape, operational state and risk policy that permits a strategy decision.	Either input alone. Neither is sufficient.	Not implemented

4.1 Why one enum would have been wrong

Merging trust and shape produces a cross-product that grows multiplicatively: five health states times five shapes is twenty-five composite states, most of which are meaningless, and every new health state multiplies the whole table again. Worse, it destroys the ability to answer *why* — "not ready" stops distinguishing "my feed dropped" from "nobody is quoting."

INDUSTRY FRAMING

Real venues publish exactly this separation. NASDAQ's ITCH protocol carries a **Trading Action** message (Halted, Quotation Only, Trading) that is entirely independent of the book's contents; a halted stock still has a well-formed book, and a trading stock can momentarily have no quotes. FIX likewise keeps **session state** (logged on, reconnecting) orthogonal to **message content**. The reason is operational: when something goes wrong at 09:31, "why did we stop trading?" must have a single unambiguous answer, and a collapsed boolean cannot give one.

4.2 Vocabulary used throughout this document

Term	Definition as used in this codebase
L3 / order-by-order	Feed granularity exposing individual orders with IDs, not just aggregated price levels (L2).
Resting order	An order sitting in the book awaiting execution. Held as a <code>RestingOrder{id, qty}</code> node.
Seed / S0	The snapshot that initialises the book before replay begins.
Checkpoint / S1	A second, independently fetched snapshot used to verify the replayed result.
Cutoff	The timestamp at which replay stops — normally the checkpoint's timestamp.
Fill credit	A recorded <code>amount_traded</code> the venue already subtracted, keyed <code>(orderId, timestamp)</code> .
Shortfall	The part of a trade's quantity <i>not</i> already covered by a fill credit. Only this is corrected.
Correction event	A synthetic <code>OrderEvent</code> the reconciler emits to fix the book. Same type as a real event.
Tombstone	A record that a correction already removed an order, so a later real remove is tolerated, not an error.
Digest	FNV-1a fingerprint over semantic fields in canonical order. Never over memory layout.
Chain link	Bitstamp's <code>event_id / pre_event_id</code> pair, used to detect stream gaps.

5 Data sources

Three inputs, two of which are streams and one of which is a photograph.

Source	Kind	Status	Role
live_orders_btcsud	WebSocket stream	PRIMARY	Every create / change / delete of an individual order. The spine of the system.
live_trades_btcsud	WebSocket stream	PRIMARY	Executions, naming both counterpart order IDs. Used to cross-check the order stream.
order_book_btcsud	REST snapshot	PRIMARY	Full book at one instant. Seeds replay (S0) and verifies it (S1).
Coinbase L2	—	UNUSED	coinbase_venue.hpp is an empty placeholder. No Coinbase code exists.

5.1 Why a snapshot is mandatory

A delta stream alone cannot build a book. If you subscribe at 10:00:00 you receive changes to orders placed at 09:00:00, whose existence you never witnessed — an `order_deleted` for an ID you never saw created. Replay therefore starts from a snapshot and applies only events strictly after it. That is why `bootstrap()` exists, and why every capture segment pairs a payload with its own seed snapshot.

A HARD-WON CAPTURE PREREQUISITE

Subscribing before fetching the snapshot is necessary but **not sufficient**. ADR 0013 records a real failure: after a reseed, the served snapshot's microtimestamp (1787875831682714) *predated* the segment's first captured event (1787875832061000) by 378 ms. Events in that window were covered by neither the snapshot nor the stream, and seven removes hit orders the book had never seen. The capture script now refetches until `snapshot.microtimestamp ≥ first captured order event`, and otherwise ends the segment with `snapshot_never_overlapped_stream` rather than emitting a seed with a hole in it.

5.2 Corpus inventory

Captures live under `data/raw/` and are gitignored. Their present state on this machine is degraded — see §2.2.

Capture	Span	Scale (ADR-recorded)	Local state
...20260822T000512Z	65 s	29,404 orders / 84 trades	payload evicted
...20260827T235604Z	856 s + 138 s	669 trades over 2 segments	frames evicted
...20260828T003754Z	—	13 tie timestamps	payload evicted
...20260809T100421Z	—	1,434 lines (legacy, order-only)	intact

6 Capture & persistence

How live bytes become a durable, replayable artifact.

6.1 The segment model

A capture is a directory containing a `manifest.json` and one or more **segments**. A segment is one replay epoch: a seed snapshot, an immutable payload of raw frames, a parallel frame index, and optionally a checkpoint snapshot. Crucially, a segment ends — and a new one begins — whenever continuity is lost.

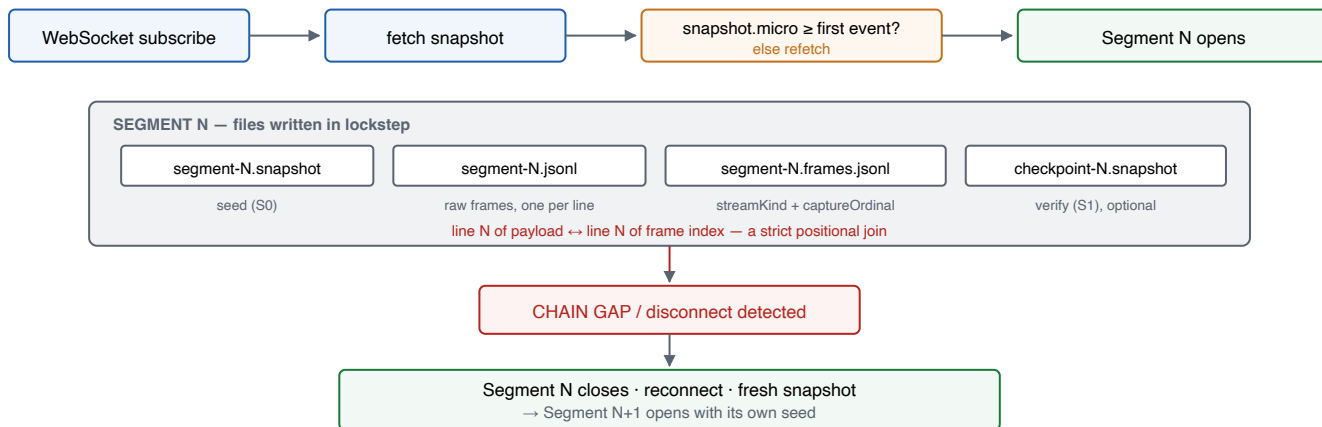


Figure 6.1 — A gap does not corrupt a capture; it ends a segment. Each segment is independently replayable from its own seed.

WHY A GAP MUST SPLIT THE SEGMENT

Bitstamp stamps each frame with `event_id` and `pre_event_id`, forming a chain. If the chain breaks, some unknown number of events were lost. Those missing events may have created, modified or deleted orders. Replaying across the break would apply later deltas to a book that no longer reflects reality, and the corruption would be *silent* — the book would still be well-formed, just wrong. Splitting into a new segment with a fresh snapshot is the only sound repair. This mirrors sequence-gap handling in exchange feeds generally: ITCH/OUCH consumers that detect a sequence discontinuity must re-request or re-snapshot rather than interpolate.

6.2 The frame index and why it is separate

The payload holds the venue's bytes *verbatim* — nothing added, nothing removed. Anything the capture process learned (which stream a frame came from, what local arrival ordinal it had) goes into a parallel file. The two are joined positionally, line N to line N.

`src/capture/segment_loader.cpp:95-131` — the positional join

```

while (true) {
    const bool hasPayloadLine = static_cast<bool>(std::getline(payloadInput, payloadLine));
    const bool hasFrameIndexLine = static_cast<bool>(std::getline(frameIndexInput, frameIndexLine));

    if (!hasPayloadLine && !hasFrameIndexLine) break;           // clean end
    if ( hasPayloadLine && !hasFrameIndexLine)
        return failure(JoinedCaptureError::frame_index_ended_early);
    if (!hasPayloadLine && hasFrameIndexLine)
        return failure(JoinedCaptureError::payload_ended_early);
    ...
    std::string_view streamKind;                                // "order" | "trade" | "control"
    std::uint64_t captureOrdinal;
  
```

Length disagreement is a hard error with its own named variant in each direction. That matters: a truncated frame index would otherwise silently shift every subsequent event onto the wrong stream kind.

SILENT-SUCCESS HAZARD

If **both** files are readable but empty, the loop breaks on its first iteration and `loadSegment` returns **success with zero events**. This is exactly the path that turned the evicted corpus (§2.2) into a confusing assertion failure rather than a clear "capture is empty" error. A non-empty-payload check would convert a silent success into a named failure.

6.3 Stream kinds

streamKind	Handling	Effect
"order"	decodeCapturedOrder()	Appends a CapturedOrderEvent
"trade"	decodeTrade()	Appends a CapturedTradeEvent
"control"	skipped silently	Subscription acks, heartbeats — recorded for provenance, not replayed
anything else	unknown_stream_kind	Whole capture rejected

6.4 The v3 binary tape PARTIAL

A fixed-width binary format exists alongside the JSONL capture: a 128-byte header (`segment_format.hpp`) plus 64-byte event records (`event_record_format.hpp`). Its header stamps the **ordering policy version** — `kOrderingPolicyOrderWinsTie = 1`, i.e. ADR 0013 — so a reader that does not recognise a policy can refuse the file rather than trusting an order it cannot reproduce.

offset 0

magic "TESEG\0\0\0" (8 bytes)

major=3 minor=1 recordSize flags

orderingPolicyVersion ← ADR 0013 = 1

InstrumentSpec · seedTimestamp · created

snapshotSha256 (32 bytes)

128

64-byte event records, fixed width

byte 0 = EventRecordType (order|trade)

bytes 1–7 = reserved, MUST be zero

byte 8.. = timestamp, then type-specific

Fixed width is what makes "seek to record N" arithmetic rather than a scan, and the **must-be-zero reserved bytes** are what make two builds produce byte-identical captures for identical input — the same discipline the legacy v2 `Record` enforces with `static_assert(sizeof(Record) == 56)` and an explicit padding check in its test.

NOT YET LOAD-BEARING

Replay does **not** consume the v3 tape. It reads JSONL and decodes on every run. The tape is a built, tested accelerator with no production caller — a real instance of infrastructure landing ahead of the need that justifies it.

7 Raw Bitstamp messages

What actually arrives on the wire, field by field.

7.1 An order frame

Taken verbatim from the committed test fixture (reformatted for width):

tests/unit/test_bitstamp_joined_capture.cpp:75 — a real order_deleted frame

```
{
  "data": {
    "id": 2037493297635328,
    "id_str": "2037493297635328",
    "order_type": 0,
    "order_subtype": 5,
    "datetime": "1786269862",
    "microtimestamp": "1786269861947000",
    "amount": 0.00171371,
    "amount_str": "0.00171371",
    "amount_traded": "0",
    "amount_at_create": "0.00171371",
    "price": 58356.1,
    "price_str": "58356.10",
    "is_liquidation": false
  },
  "channel": "live_orders_btccusd",
  "event": "order_deleted",
  "event_id": "0006589a-5c98-2678-0000-000101000020",
  "pre_event_id": "0006589a-5c97-f798-0000-000100000020",
  "order_source": "orderbook"
}
```

Note the deliberate redundancy: **every numeric field appears twice**, once as a JSON number and once as a string. That redundancy is load-bearing, and §7.3 explains why.

7.2 Field-by-field mapping

Raw field	Raw type	Parsed by	Internal type	Final member
event	string	string compare	EventKind	event.kind
data.id_str	string	parseInteger()	OrderId	event.order_id
data.id	uint64	get_uint64()	—	<i>cross-check only</i>
data.microtimestamp	string	parseInteger()	uint64	event.venue_timestamp_us
data.order_type	uint64	0→buy, 1→sell	Side	event.side
data.price_str	decimal string	parseDecimal(·,2)	Price	event.price
data.amount_str	decimal string	parseDecimal(·,8)	Qty	event.quantity
data.amount_traded	decimal string	parseDecimal(·,8)	Qty	amountTraded (<i>sidecar</i>)
event_id / pre_event_id	string	decodeChain()	ChainLink	<i>gap detection</i>
amount / price	float	deliberately ignored — see §9		
order_subtype, is_liquidation, datetime	—	not decoded; no current consumer		

7.3 The id vs id_str tripwire

The decoder parses `id_str` exactly, then parses the numeric `id` independently and **requires them to agree**:

src/feed/bitstamp/decoder.cpp:173–184

```
// ADR 0005: the duplicate representations must agree. Bitstamp order IDs are around
// 2.0e15 today against a 9.0e15 ceiling (2^53), above which the JSON number silently
// loses precision while id_str stays exact. This is the tripwire for that day; it has
// never fired across 619,803 captured order events.
std::uint64_t numeric_id{};
if (doc["data"]["id"].get_uint64().get(numeric_id))
    return failure(DecoderError::missing_field);
if (numeric_id != *decoded_id)
    return failure(DecoderError::id_mismatch);
```

JSON numbers are IEEE-754 doubles in most parsers. A double holds integers exactly only up to $2^{53} \approx 9.0 \times 10^{15}$. Bitstamp IDs currently sit near 2.0×10^{15} — the same order of magnitude. The day IDs cross that ceiling, `id` starts rounding while `id_str` stays exact, and this check fails loudly instead of silently merging two different orders into one book entry.

WHY THIS PATTERN RECURS

Twitter/X hit exactly this wall in 2010 — "snowflake" IDs exceeded 2^{53} and every JavaScript client silently corrupted them, which is why their API still returns both `id` and `id_str`. Bitstamp's dual representation is the same defensive move. The lesson this codebase takes from it: when a protocol hands you redundancy, treat agreement as an *invariant to assert*, not noise to discard.

7.4 A trade frame

tests/unit/test_bitstamp_joined_capture.cpp:78

```
{
  "data": {
    "id": 619501668,
    "timestamp": "1787174919",
    "amount": 0.34778488,
    "amount_str": "0.34778488",
    "price": 69590.13,
    "price_str": "69590.13",
    "type": 0,
    "microtimestamp": "1787174919977000",
    "buy_order_id": 2041200416022642,
    "sell_order_id": 2041200408317957,
    "channel": "live_trades_btccusd",
    "event": "trade"
  }
}
```

AN INCONSISTENCY WORTH KNOWING

`buy_order_id` and `sell_order_id` are **bare JSON integers** here — no `_str` variant exists, unlike order frames. The code comments this explicitly at `trade_decoder.cpp:42–43`. So the 2^{53} protection available on the order stream is *not* available on the trade stream. If IDs ever cross that ceiling, trade reconciliation would begin matching wrong orders with no tripwire to catch it.

Note also what is *not* decoded: the trade's price. The comment at `trade_decoder.cpp:13` states the reason — reconciliation matches on ID and quantity only, so price would be dead weight in the normalized type.

8 Parsing & decoding

Two decode entry points, and the one that survived.

8.1 The two order decoders

Function	Returns	Status	Used by
decodeCapturedOrder()	DecodedCapturedOrder {event, amountTraded}	CURRENT	segment_loader.cpp:133 — the live replay path
decodeOrder()	OrderEvent	LEGACY	The v2 recorder path only. Kept permanently: that path needs order-only decoding.
decodeTrade()	TradeEvent	CURRENT	segment_loader.cpp:145
decodeChain()	ChainLink	CURRENT	Gap detection in capture

The two order decoders are **near-identical bodies** — roughly 115 lines duplicated, differing only in the result type and the extra `amount_traded` extraction. That duplication is the visible cost of ADR 0013's decision to keep the fill quantity out of `OrderEvent` (§10.3). It is deliberate, but it is real: a fix to the side-parsing logic must be applied twice.

8.2 Decode error taxonomy

Error	Trigger	Meaning
malformed_json	<code>parser.iterate()</code> fails	Not JSON at all — truncated frame or corrupt file
not_order_event	<code>event</code> is unrecognised	Normal: subscription acks and heartbeats land here
missing_field	Field absent or wrong JSON type	Protocol change, or wrong stream kind in the frame index
invalid_field	Field present but unparseable	Bad decimal, out-of-range side code, overflow
id_mismatch	<code>id != id_str</code>	The 2 ⁵³ tripwire (§7.3)

8.3 A lifetime hazard the code guards explicitly

src/feed/bitstamp/decoder.cpp:49–51 and 166–168

```
// Keep Result alive while using valueIf(); calling it on a temporary would dangle.
const Result<uint64_t, ParseError> id_result = parseInteger(id_str);
const uint64_t* decoded_id = id_result.valueIf();
```

`Result::valueIf()` returns a **pointer into the Result's own `std::variant`**. Writing `parseInteger(x).valueIf()` would bind that pointer to a temporary destroyed at the end of the full expression — a dangling read that would usually "work" in testing and fail unpredictably later. Naming the `Result` keeps it alive. The same discipline appears in `snapshot.cpp:38` with an identical comment.

DESIGN TENSION WORTH NOTING

This is the price of a pointer-based accessor. `std::expected` (C++23) and Rust's `Result` return *references or values* through APIs that make the lifetime relationship explicit to the compiler; a raw `const T*` from `get_if` cannot. The codebase chose `get_if` to keep querying the inactive alternative non-throwing (ADR 0002 forbids exceptions in this path), and pays for it with a rule humans must remember. That is a legitimate trade, but it is a trade.

8.4 simdjson On-Demand and forward-only reads

The parser is simdjson's On-Demand API, which does not build a DOM — it walks the buffer lazily and materialises values as you ask for them. The constraint is that iteration is **forward-only**. `snapshot.cpp` handles this explicitly:

src/feed/bitstamp/snapshot.cpp:14–15

```
// On Demand is forward-only, so consume each row's three fields in one pass.
static Result<BookSnapshot, SnapshotError> readRowFields(simdjson::ondemand::value row_result,
                                                         std::string_view out[3]) {
```

Each snapshot row `["price", "amount", "id"]` is read once into a small array rather than indexed three times. Re-indexing would either fail or silently re-scan.

MINOR: AN ODD RETURN TYPE

`readRowFields` returns `Result<BookSnapshot, SnapshotError>` but its success value is always a default-constructed empty `BookSnapshot{}` that the caller discards. It is being used as a `Result<void, E>`. Harmless, but it costs a pointless construction per row and misleads on first read.

9 Exact-integer conversion

The single most important correctness decision in the codebase.

9.1 Why not `double`

Binary floating point cannot represent most decimal fractions. $0.1 + 0.2 \neq 0.3$ is the famous demonstration; the consequence in a trading system is that a book's aggregate quantity drifts from the sum of its orders after enough arithmetic, and two runs can disagree depending on operation order. Prices are therefore stored as `int64` in **venue ticks**: "58356.10" with `price_decimals = 2` becomes exactly 5835610.

Quantities use 8 decimals (satoshis): "0.00171371" becomes 171371. The scale comes from caller-supplied `InstrumentSpec`, never inferred per message — inference would make the same string mean different amounts on different frames.

Raw string	Scale	Stored int64	Reasoning
"58356.10"	2	5835610	Price in cents
"0.00171371"	8	171371	Quantity in satoshis
"0"	8	0	No fill on this event
"2037493297635328"	—	2037493297635328	Order ID via <code>parseInteger</code> , unscaled

9.2 Overflow guarded *before* the operation, never after

src/core/text_to_int.cpp:41-52 — the integer-digit loop

```
int x = static_cast<int>(text[i] - '0');
// Check before operating, never after: a wrapped int64 can land back under any
// later range check, so a post-hoc test passes on already-corrupt data. Signed
// overflow is also UB in its own right, so the multiply must not happen at all.
if (parsed_text_before_decimal > INT64_MAX / 10)
    return failure(te::ParseError::overflow);
parsed_text_before_decimal *= 10;
if (parsed_text_before_decimal > INT64_MAX - x)
    return failure(te::ParseError::overflow);
parsed_text_before_decimal += x;
```

Two distinct reasons are packed into that comment, and both matter:

1. **A post-hoc check is unsound.** Wrapping is modular, so an overflowed value can land back inside a plausible range. `if (result > MAX) reject;` never fires.
2. **Signed overflow is undefined behaviour.** The compiler is entitled to assume it cannot happen and optimise the check away entirely. The multiply must not execute at all.

9.3 Parse error taxonomy

ParseError	Trigger
empty_input	Zero-length string
invalid_character	Anything not 0-9 or a single .
invalid_format	Second decimal point, or the string is just "."
negative_not_allowed	A - anywhere. Prices and quantities are non-negative here.
excess_precision	More fractional digits than the scale allows — refuses to round silently
unsupported_scale	Scale > 18 (would overflow the multiplier)
overflow	Any guarded operation would exceed INT64_MAX

`excess_precision` is the interesting one. "0.123456789" at scale 8 is **rejected**, not rounded. Rounding would silently invent or destroy value; a venue sending more precision than declared means the `InstrumentSpec` is wrong, and that is worth an error.

9.4 The architecture guard

Discipline that depends on remembering is discipline that eventually fails. The project enforces this one in CI: per `TOD0.md`'s closed items, "clock and floating-point architecture guards are active in CI and have deliberately broken tests" — a test that *introduces* a `double` must fail the build, proving the guard fires rather than merely existing.

INDUSTRY FRAMING

Fixed-point integer money is universal in financial infrastructure. FIX transmits prices as decimal strings for exactly this reason; SQL uses `DECIMAL/NUMERIC` rather than `FLOAT` for currency; the JVM world uses `BigDecimal`; and exchange matching engines work in integer ticks throughout. The convention of **storing the smallest unit as an integer** (cents, satoshis, pips) is old enough to predate computers — double-entry bookkeeping never had fractional pennies either.

10 The normalized event model

Fixed-size, venue-neutral, trivially copyable — and why each of those is enforced.

10.1 The core types

```
include/te/core/types.hpp + include/te/feed/events.hpp
```

```
enum class Side      : std::uint8_t { buy, sell };
enum class EventKind: std::uint8_t { add, modify, remove };

struct Price { std::int64_t ticks{}; }; // exact venue ticks
struct Qty   { std::int64_t units{}; }; // signed: differences must not wrap
struct OrderId { std::uint64_t value{}; };

struct OrderEvent {
    std::uint64_t venue_timestamp_us{};
    OrderId      order_id{};
    Price        price{};
    Qty          quantity{};           // RESULTING resting qty, never a delta
    Side         side{};
    EventKind     kind{};
};
```

Type	Representation	Why that representation	Created by / consumed by
Price	int64 ticks	Exact; scale lives in InstrumentSpec (ADR 0004)	decoder → book, digest
Qty	int64 signed	Unsigned subtraction wraps; quantity differences are computed constantly	decoder → level aggregates
OrderId	uint64	Parsed exactly from id_str (ADR 0005)	decoder → index, reconciler
Side	uint8 enum	One byte is part of the fixed record layout	decoder → side selection
EventKind	uint8 enum	Same; also drives apply() dispatch	decoder → apply()
OrderEvent	40 bytes, trivially copyable	Queue transport + on-disk record embed it	decoder → book, reconciler, tape

10.2 Strong types that refuse to mix

Price and Qty are both "an int64 in a struct". Wrapping them separately means Price cannot be passed where Qty is expected — the compiler rejects it. A bare int64_t for both would compile qtyAt(side, quantity) happily.

The cost is visible at every use: price.ticks, qty.units. The benefit is a class of bug that cannot occur. This is the classic **strong typedef** / newtype pattern — std::chrono is the canonical C++ example, where seconds and milliseconds are distinct types precisely so unit confusion is a compile error rather than a Mars Climate Orbiter.

10.3 quantity is a level, not a delta

```
include/te/feed/events.hpp:27
```

```
// Resulting resting quantity, never a delta. Remove locates by ID and ignores this value.
Qty quantity{};
```

This single sentence drives much of the system's behaviour. Bitstamp reports the **post-change remaining amount**, so an order_changed already has any fill subtracted. That is exactly why naïve trade correction double-counts (§21), and why applyModify can set rather than adjust. It is also why remove ignores the incoming price and quantity entirely and trusts its own stored locator (§18).

10.4 What is deliberately not in OrderEvent

Excluded	Carried instead as	Reason
amount_traded	DecodedCapturedOrder.amountTraded, then CapturedOrderEvent	ADR 0013: adding it takes Record from 56→64 bytes and forces an on-disk format bump for data the book never reads
event_id / pre_event_id	ChainLink via decodeChain()	Capture-time gap detection; irrelevant to book mutation
captureOrdinal	CapturedOrderEvent.captureOrdinal	Local arrival provenance. Explicitly not the tie-break (§14.3)
order_subtype, is_liquidation	not decoded	No consumer exists

THE ENVELOPE PATTERN

`CapturedOrderEvent` wraps the pure `OrderEvent` with capture metadata:

```
struct CapturedOrderEvent { OrderEvent event; Qty amountTraded; std::uint64_t captureOrdinal{}; };
struct CapturedTradeEvent { TradeEvent event;                std::uint64_t captureOrdinal{}; };
```

The header notes that keeping these envelopes *outside* the disk loader lets replay consume events from any source able to preserve arrival order — not just files. That is what would make a live feed reuse the identical replay path.

10.5 The static assertions as executable documentation

`types.hpp` and `events.hpp` carry assertions whose messages explain the constraint rather than restate it:

```
static_assert(std::is_trivially_copyable_v<OrderEvent>,
              "OrderEvent must remain trivially copyable for fixed-capacity queue transport.");
static_assert(std::is_same_v<std::underlying_type_t<Side>, std::uint8_t>,
              "Side must retain its one-byte representation because normalized event and "
              "fixed-capacity queue layouts depend on it.");
```

Adding a `std::string` to `OrderEvent` breaks the build at the definition with an explanation of *why*, rather than silently degrading a lock-free queue into something that allocates. The assertions are careful to add that trivial copyability "does not define its binary serialization" — the on-disk format is specified explicitly by `event_record_format.hpp`, never by `memcpy` of the struct.

11 Snapshot & bootstrap

Why replay never begins with an empty book.

11.1 Snapshot parsing

A Bitstamp snapshot is a JSON document with a `microtimestamp` and two arrays of `[price, amount, id]` triples. `parseSnapshot()` converts each row into a `SnapshotOrder`, tagging side by which array it came from.

src/feed/bitstamp/snapshot.cpp:94-129 — table-driven side handling

```
struct SideSource { const char* key; Side side; SnapshotError missing_error; };
const SideSource sources[2] = {
    {"bids", Side::buy, SnapshotError::missing_bids},
    {"asks", Side::sell, SnapshotError::missing_asks},
};
for (const SideSource& source : sources) {
    ...
    if (!seen_ids.insert(order.order_id).second)
        return failure(SnapshotError::duplicate_order_id); // whole snapshot rejected
```

A duplicate ID rejects the **entire snapshot**, not just the row. The comment states the reasoning: duplicates make the seed ambiguous. If the same order appeared at two prices, there would be no defensible way to choose, and the resulting book would be wrong in a way no later event could repair.

11.2 Bootstrap: a snapshot is replayed as synthetic adds

src/feed/bitstamp/bootstrap.cpp:7-31 — complete function

```
Result<OrderBook, ApplyError> bootstrap(BookSnapshot book, TradeReconciler* reconciler) {
    OrderBook orderBook;
    for (size_t i{}; i < book.orders.size(); ++i) {
        SnapshotOrder snapshotOrder = book.orders.at(i);
        const OrderEvent& orderEvent{
            book.microtimestamp, snapshotOrder.order_id, snapshotOrder.price,
            snapshotOrder.quantity, snapshotOrder.side, EventKind::add,
        };
        const auto result = orderBook.apply(orderEvent);
        if (!result.hasValue()) return failure(*result.errorIf());

        // Seed the shadow only after the same order has successfully entered the real book.
        if (reconciler) reconciler->observe(orderEvent, Qty{});
    }
    orderBook.validateStructure();
    return success(std::move(orderBook));
}
```

Three design choices are visible here:

- **No special path.** Every snapshot order becomes an ordinary `EventKind::add` through the ordinary `apply()`. The book has exactly one insertion path, so bootstrap cannot drift from replay.
- **All orders share the snapshot's timestamp.** This is what makes the seed a single instant, and it is what `MergeCursor` later uses as its exclusive lower bound.
- **Shadow follows reality.** The reconciler is told about an order only *after* the real book accepted it, so the shadow can never contain something the book does not.

MINOR ISSUES

`const OrderEvent&` binds a reference to a temporary aggregate — legal (lifetime is extended) but needlessly indirect; a plain value would be clearer and identical in cost. `book.orders.at(i)` also copies each `SnapshotOrder` per iteration where a `const&` range-for would not. Neither affects correctness.

11.3 The replay window



Figure 11.1 — The window is half-open: $\text{seed} < ts \leq \text{cutoff}$. Events outside it are counted so the three regions sum to the input size.

Pre-seed handling is subtle and worth the detail. Pre-seed *orders* are not applied — the snapshot already reflects them — but they are still fed through the classifier:

`src/feed/bitstamp/replay.cpp:143-149`

```
// Pre-seed orders still train the stateful classifier (notably price-zero lifecycles), but
// never touch the seeded book. Pre-seed trades are already represented by the snapshot.
for (std::size_t index = 0; index < cursor.ordersBeforeSeed(); ++index) {
    classifier.classify(orderEvents.at(index).event);
    ++replayStats.orderEventsBeforeSeed;
}
replayStats.tradeEventsBeforeSeed = cursor.tradesBeforeSeed();
```

The classifier is **stateful** (§12). If a price-zero order was created before the seed and deleted after it, skipping the pre-seed create would leave the classifier ignorant of that lifecycle, and the post-seed delete would be treated as an ordinary event against an order the book never held. Training on the prefix prevents that.

12 Classification

The filter that decides what is even eligible to touch the book.

src/feed/bitstamp/classifier.cpp – complete implementation

```
EventDisposition EventClassifier::classify(const OrderEvent& event) {
    // A price-zero create opens a lifecycle that never belongs in the book. Remember its ID
    // because later messages in the same lifecycle otherwise look like ordinary orders.
    if (event.kind == EventKind::add) {
        if (event.price == Price{0}) {
            zeroPriceOrders_.insert(event.order_id);
            ++stats_.zeroPriceLifecycle;
            return EventDisposition::zero_price_lifecycle;
        }
        ++stats_.appliedToBook;
        return EventDisposition::apply_to_book;
    }

    // Filter the entire remembered lifecycle, even when later messages carry a plausible price.
    const auto tracked = zeroPriceOrders_.find(event.order_id);
    if (tracked != zeroPriceOrders_.end()) {
        ++stats_.zeroPriceLifecycle;
        if (event.kind == EventKind::remove) zeroPriceOrders_.erase(tracked); // terminal
        return EventDisposition::zero_price_lifecycle;
    }

    ++stats_.appliedToBook;
    return EventDisposition::apply_to_book;
}
```

12.1 Why statefulness is unavoidable here

A market order arriving at Bitstamp can appear on `live_orders` with `price_str: "0"` — it has no limit price. It never rests. The obvious filter is "drop events with price zero", and it is wrong: **later events in the same lifecycle can carry a real price**. A stateless filter would drop the create and then accept the modify, producing an `unknown_order_id` failure — or worse, an add that never should have existed.

So the classifier remembers IDs. The `remove` is treated as terminal and releases the ID, which also bounds memory: the set holds only currently-open zero-price lifecycles.

Input	Disposition	Effect on <code>zeroPriceOrders_</code>
add, price = 0	<code>zero_price_lifecycle</code>	ID inserted
add, price > 0	<code>apply_to_book</code>	unchanged
modify, ID tracked	<code>zero_price_lifecycle</code>	ID retained
remove, ID tracked	<code>zero_price_lifecycle</code>	ID erased (terminal)
modify / remove, untracked	<code>apply_to_book</code>	unchanged

AN ID-REUSE GAP

The classifier releases an ID only on `remove`. `Replay` separately handles ID reuse for its correction tombstones — `replay.cpp:76–79` erases a tombstone when a fresh `add` arrives for the same ID, with the comment "Order IDs may be reused; a new lifecycle invalidates any old removal tombstone." The classifier has no equivalent: a price-zero lifecycle that never receives its `remove`, followed by ID reuse for a real order, would see the new order silently filtered. Whether Bitstamp can produce that is unverified. Note also that a price-zero `add` for an *already-tracked* ID re-inserts without clearing, which is consistent but untested.

13 Two streams, one sequence

The ordering problem, stated before the solution.

Orders and trades arrive on separate WebSocket channels. Each is internally time-ordered. Neither is ordered relative to the other, and the same real-world event — an execution — appears in **both**: as a quantity reduction on `live_orders`, and as a trade on `live_trades`.

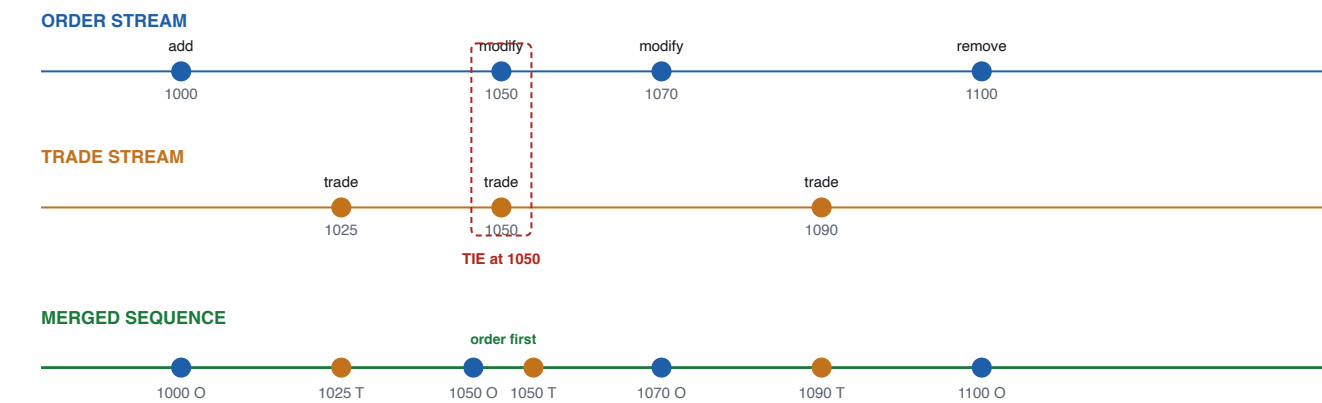


Figure 13.1 — Two individually-ordered streams become one sequence. At the shared timestamp 1050 the **order** is processed first.

13.1 Why the tie-break cannot be arbitrary

ADR 0013 documents both failure directions with measured evidence, and neither ordering is correct for both:

Scenario at a shared timestamp	Order-first	Trade-first
<code>order_changed</code> carrying its own fill	Double-counts: the venue already subtracted the fill, then the reconciler subtracts it again	Correct
<code>order_created</code> consumed by a trade at the same instant	Correct: <code>observe()</code> records the order before <code>reconcile()</code> looks for it	Reconciler finds nothing, emits no correction; the order rests forever

The resolution was not to pick a better tie-break but to **remove the dependency**: make corrections idempotent so ordering stops being load-bearing for correctness (§21). The tie-break then survives purely as a determinism rule.

14 MergeCursor

Forty lines that own one decision, and nothing else.

14.1 The whole ordering policy, in one comparison

src/feed/merge_cursor.cpp:32-47 — next()

```
std::optional<MergedPick> MergeCursor::next() {
    const bool orderAvailable = orderInWindow();
    const bool tradeAvailable = tradeInWindow();
    if (!orderAvailable && !tradeAvailable) return std::nullopt;

    // The whole tie-break lives in this one comparison: `<=` hands a shared timestamp to the order
    // event. Nothing else in the project is allowed to decide this.
    if (orderAvailable && (!tradeAvailable ||
        orderEvents_[orderPtr_].event.venue_timestamp_us <=
        tradeEvents_[tradePtr_].event.venue_timestamp_us)) {
        return MergedPick{MergedStream::order, orderPtr_++};
    }
    return MergedPick{MergedStream::trade, tradePtr_++};
}
```

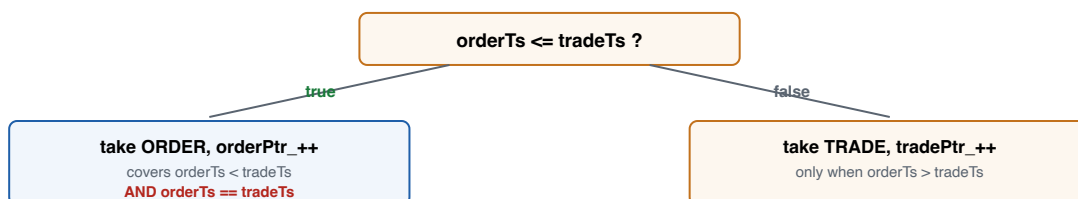


Figure 14.1 — The single character that encodes ADR 0013. Changing `<=` to `<` silently reverses the policy.

14.2 Deliberately ignorant

The class header is explicit about what this component must *not* know:

include/te/feed/merge_cursor.hpp:22-32

```
// The cursor owns the ordering decision and nothing else: no book, no classifier, no reconciler.
// That is the point. Live replay and the v3 tape writer both need ADR 0013's tie-break, and a
// policy written twice is a policy that eventually disagrees with itself.
```

Two consumers need this policy: the replay controller and the tape writer. Had each implemented its own merge, the tape could be written in one order and replayed in another, and the disagreement would surface as an unreproducible book weeks later. Centralising it means the policy has exactly one definition and one place to test.

INDUSTRY FRAMING

This is a **single point of truth for a cross-cutting policy** — the same reason distributed databases centralise their comparator/collation rules, and the reason Kafka Streams forces you to declare a timestamp extractor once per topology rather than per operator. Ordering policy is contagious: every component that re-derives it becomes a place the system can disagree with itself.

14.3 Why `captureOrdinal` is carried but not used for ordering

The plan originally declared the replay key as `(venueTimestampMicros, captureOrdinal)`. Measurement rejected it. ADR-recorded figures across four segments:

Capture segment	Timestamps carrying both	Ordinal puts trade first
20260822T000512Z seg 0	61	58
20260827T235604Z seg 0	327	304
20260827T235604Z seg 1	26	20
20260828T003754Z seg 0	13	12
total	427	394 (92%)

Bitstamp's trade frame reaches the socket *before* the matching order frame 92% of the time. Ordering by arrival ordinal therefore processes the trade first — precisely the hazard the tie-break exists to prevent. Running both policies:

Policy	Final book digest	Checkpoint mismatches	Apply failures
order wins tie (current)	1efdb47806f2763f	0	0
(venue_ts, captureOrdinal)	1efdb47806f2763f	0	41

The final books agree — each spurious correction is cancelled by the raw order event that then fails as redundant — but `Replay` aborts on the first of those 41 failures. Adopting the ordinal would require *tolerating* failures, which is the silent-corruption posture the project rejects. The ordinal is still decoded and carried as provenance; the evidence above is only measurable because it is.

14.4 Window accounting

The constructor advances both pointers past the seed, recording how many it skipped. After exhaustion, whatever remains is past the cutoff. The three counts must sum to the input size:

```
std::size_t orderEventsAccountedFor() const {
    return orderEventsBeforeSeed + orderEventsRead + orderEventsAfterCutoff;
}
```

`replay.hpp:16–18` states the purpose plainly: "without the two window counters a merge-loop bug that silently skipped events would leave no trace." This is a **conservation invariant** — the same instinct as double-entry bookkeeping, where the check is not that any single number is right but that the totals must balance.

15 OrderBook internal architecture

Two views of the same orders, and why both are required.

15.1 The three containers

include/te/book/order_book.hpp:102–104

```
std::map<Price, PriceLevel>      bids_{};
std::map<Price, PriceLevel>      asks_{};
std::unordered_map<OrderId, OrderLocator, OrderIdHash> orderIndex_{};
```

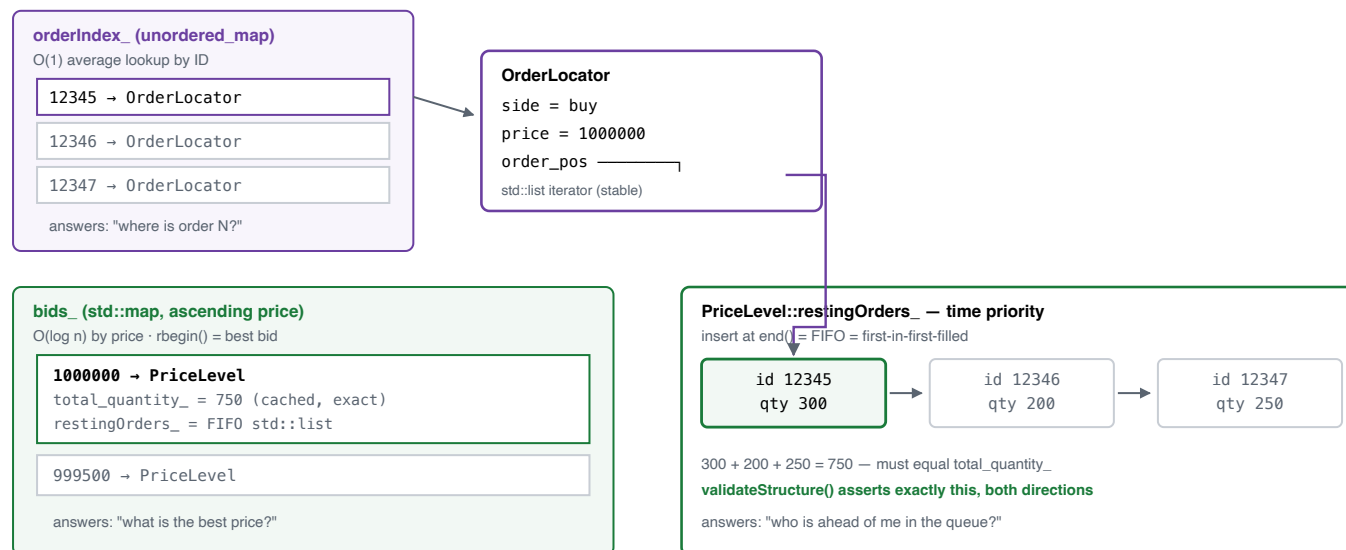


Figure 15.1 — `orderIndex_[12345] → OrderLocator{buy, 1000000, iterator} → the exact list node`. Two access paths, one set of orders.

15.2 Why both structures are necessary

Question	Answered by	Cost	Without it
"What is the best bid?"	<code>bids_.rbegin()</code>	$O(1)$	Scan every order
"How much rests at price P?"	<code>levels.find(P) → cached total</code>	$O(\log n)$	Sum the level's list every query
"Where is order 12345?"	<code>orderIndex_.find(id)</code>	$O(1)$ avg	Scan every level and every list — $O(n)$
"Who is ahead of order 12345?"	Its position in the FIFO list	—	Impossible: L2 books cannot answer this

A delete message names only an ID. With price-ordered maps alone, finding that order means scanning the entire book — and deletes are the majority of L3 traffic. With the index alone, you could find any order instantly but could not answer "what is the best price?" without scanning. Both structures exist because the workload genuinely asks both kinds of question.

15.3 Why the book is move-only

include/te/book/order_book.hpp:54–60

```
// OrderLocator stores iterators into this book's PriceLevel lists. A normal copy would copy
// those iterators while copying the lists into different nodes, leaving the copied locators
// referring to the original book. Moving transfers ownership of the existing nodes instead.
OrderBook(const OrderBook&) = delete;
OrderBook& operator=(const OrderBook&) = delete;
OrderBook(OrderBook&) = default;
OrderBook& operator=(OrderBook&) = default;
```

A default copy would duplicate the lists (new nodes at new addresses) while copying the locators *verbatim* — so every locator in the copy would point into the **original** book. Reads would appear to work. Mutating the copy would corrupt the original. It is the worst class of bug: silent, delayed and non-local. Deleting the copy operations makes it a compile error, and four `static_assert`s pin the guarantee so nobody re-enables it accidentally.

`std::list` is chosen precisely because its iterators remain valid across insertions and unrelated erasures. A `std::vector` would invalidate every stored locator on reallocation.

15.4 Complexity summary

Operation	Complexity	Dominant cost
<code>applyAdd</code>	$O(\log L)$	<code>map::try_emplace</code> ; list insert and index emplace are $O(1)$
<code>applyModify</code> (qty only)	$O(1)$ avg + $O(\log L)$	Index lookup, then level lookup; node untouched
<code>applyModify</code> (price change)	$O(\log L)$	Two level lookups + insert + erase
<code>applyRemove</code>	$O(1)$ avg + $O(\log L)$	Index lookup, then level lookup, then $O(1)$ list erase
<code>bestBid</code> / <code>bestAsk</code>	$O(1)$	<code>rbegin()</code> / <code>begin()</code>
<code>qtyAt</code>	$O(\log L)$	Level lookup; total is cached, not summed
<code>digest</code>	$O(L)$	Walks every non-empty level once
<code>validateStructure</code>	$O(N + L)$	Walks every order twice. Debug-only for this reason.

L = distinct price levels, N = total resting orders.

THIS IS THE REFERENCE, NOT THE FAST PATH

The header calls `OrderBook` "the correctness oracle; optimized books must reproduce its results (ADR 0007/0012)." `std::map` and `std::list` are node-based and cache-hostile — a production book would use an array of price levels indexed by tick offset with intrusive order nodes. The point of this implementation is to be *obviously correct* so a faster one can be checked against it via `digest()` .

16 ADD lifecycle

`order_created` → `EventKind::add` → `applyAdd()`

16.1 The gatekeeper in `apply()`

Before any dispatch, `apply()` performs two checks that belong to every event kind. Doing them once here rather than three times in the handlers is what keeps the ID-state contract consistent.

`src/book/order_book.cpp:7-46` — `apply()`

```
if (orderEvent.side != Side::buy && orderEvent.side != Side::sell)
    return failure(ApplyError::invalid_side);

// Reject ID-state contradictions before selecting a side or touching a price level.
if (orderIndex_.contains(orderEvent.order_id)) {
    if (orderEvent.kind == EventKind::add)
        return failure(ApplyError::duplicate_order_id);
} else {
    if (orderEvent.kind == EventKind::modify || orderEvent.kind == EventKind::remove)
        return failure(ApplyError::unknown_order_id);
}

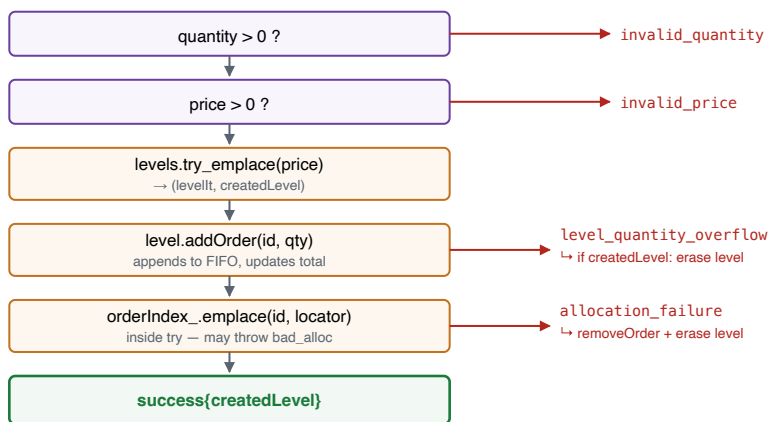
const auto result = [&]() -> Result<ApplyOutcome, ApplyError> {
    switch (orderEvent.kind) {
        case EventKind::add:    return applyAdd(orderEvent);
        case EventKind::modify: return applyModify(orderEvent);
        case EventKind::remove: return applyRemove(orderEvent);
    }
    return failure(ApplyError::invalid_event_kind); // reached only on a cast-in bad enum
}();

#ifdef NDEBUG
if (result.hasValue()) validateStructure(); // full sweep, debug builds only
#endif
return result;
```

Because `apply()` has already proven the ID exists, `applyModify` and `applyRemove` can `assert` rather than branch on their lookup — the condition is an established precondition, not an input to re-validate.

The immediately-invoked lambda exists so the debug validation runs on every success path without repeating it at each `return`. The trailing `return` after a `switch` covering all enumerators handles a value cast in from outside the enum's range — defensive, and the reason `invalid_event_kind` exists at all.

16.2 `applyAdd()` step by step



STRONG EXCEPTION GUARANTEE — every failure path undoes exactly what it did

A level created solely for a failed insertion is erased. An order inserted before a failed index `emplace` is removed, then its level erased if it was new. The book is never left half-mutated.

Figure 16.1 — `applyAdd()`. Note that both mutation steps have a matching rollback.

16.3 Rollback, exactly

`src/book/order_book.cpp:56-84`

```

auto& levels = (orderEvent.side == Side::buy) ? bids_ : asks_;
auto [levelIt, createdLevel] = levels.try_emplace(orderEvent.price);
const std::optional<OrderHandle> orderHandle =
    levelIt->second.addOrder(orderEvent.order_id, orderEvent.quantity);
if (!orderHandle.has_value()) {
    // Roll back a level created solely for this failed insertion.
    if (createdLevel) levels.erase(levelIt);
    return failure(ApplyError::level_quantity_overflow);
}

const OrderLocator orderLocator{
    .side = orderEvent.side, .price = orderEvent.price, .order_pos = *orderHandle,
};
try {
    orderIndex_.emplace(orderEvent.order_id, orderLocator);
} catch (const std::bad_alloc&) {
    levelIt->second.removeOrder(*orderHandle); // undo the list insert
    if (createdLevel) levels.erase(levelIt); // undo the level creation
    return failure(ApplyError::allocation_failure);
}

```

The `createdLevel` flag from `try_emplace` is what makes this precise. If the level already existed with other orders resting on it, erasing it would destroy unrelated state; the flag distinguishes "I created this" from "I found this."

INDUSTRY FRAMING

This is Abrahams' **strong exception guarantee**: an operation either completes or leaves state exactly as it found it. It is the same contract `std::vector::push_back` provides, and the same idea as a database transaction's atomicity. In a market-data reconstructor it matters more than usual — a half-applied event produces a book that is structurally valid but *quietly wrong*, and every subsequent event compounds the error.

REMINDER OF FINDING 3

Immediately after the success return, `order_book.cpp:88` carries an unreachable duplicate `orderIndex_.emplace(...)`. It does not execute, but it reads as if insertion happens outside the `try` — the opposite of the rollback design above.

16.4 PriceLevel::addOrder

src/book/price_level.cpp:7-21

```

std::optional<OrderHandle> PriceLevel::addOrder(OrderId id, Qty quantity) {
    if (quantity.units <= 0) return std::nullopt;
    if (total_quantity_.units > std::numeric_limits<std::int64_t>::max() - quantity.units)
        return std::nullopt; // guard BEFORE adding

    auto orderIt = restingOrders_.insert(restingOrders_.end(), RestingOrder{id, quantity});
    total_quantity_.units += quantity.units;
    return orderIt;
}

```

`insert(end(), ...)` is what establishes **time priority**: new orders go to the back of the queue at their price. That is the "time" half of price-time priority, the matching rule used by most major exchanges, and it is the property an L2 book fundamentally cannot represent.

16.5 Before / after

BEFORE

BIDS
 100.00 → [A: 2 BTC]
ASKS
 101.00 → [B: 1 BTC]

INCOMING

BUY 1 BTC @ 99.50
 order id 123

AFTER

BIDS
 100.00 → [A: 2 BTC]
 99.50 → [123: 1 BTC] ← new level
ASKS
 101.00 → [B: 1 BTC]

orderIndex_.change

A → {buy, 10000, iter}
 B → {sell, 10100, iter}

A → {buy, 10000, iter} B → {sell, 10100, iter}
 123 → {buy, 9950, iter} ← inserted
 ApplyOutcome{ createdLevel = true, removedLevel = false }

Figure 16.2 — Prices shown in display form; stored as integer ticks (99.50 → 9950 at scale 2).

17 MODIFY lifecycle

`order_changed` → `EventKind::modify` → two genuinely different cases

17.1 Case A — quantity only, price unchanged

src/book/order_book.cpp:113–122

```
if (orderEvent.price == locator.price) {
    // A quantity-only change keeps the existing list node and therefore its position.
    const bool quantityChanged =
        oldLevelIt->second.changeQty(locator.order_pos, orderEvent.quantity);
    if (!quantityChanged) return failure(ApplyError::level_quantity_overflow);
    return success(ApplyOutcome{});
}
```

The list node is mutated in place. Its position in the FIFO is untouched, so **queue priority is preserved** — and the locator needs no update, because neither side, price nor node identity changed. `ApplyOutcome{}` is all-true: no level created, none removed.

This models real exchange behaviour: reducing an order's size normally keeps its place in the queue, whereas increasing it or changing its price normally does not. Note this implementation does not distinguish increase from decrease — both keep position. Whether Bitstamp's semantics match is not asserted anywhere in the code.

src/book/price_level.cpp:29–44 — `changeQty`

```
bool PriceLevel::changeQty(OrderHandle orderHandle, Qty newQty) {
    assert(total_quantity_.units >= orderHandle->qty.units);
    if (newQty.units <= 0) return false;

    const std::int64_t withoutOld = total_quantity_.units - orderHandle->qty.units;
    // Compute the candidate total before mutating either the aggregate or the order node.
    if (withoutOld > std::numeric_limits<std::int64_t>::max() - newQty.units) return false;

    total_quantity_.units = withoutOld + newQty.units;
    orderHandle->qty = newQty;
    return true;
}
```

Both mutations happen only after the overflow check. Had the aggregate been updated first and the node second, an overflow return would leave the cached total disagreeing with the sum of its orders — an invariant violation that `validateStructure()` would catch in debug, and that would silently corrupt digests in release.

17.2 Case B — price change: insert before remove

src/book/order_book.cpp:124–144

```
// Insert at the destination first. If it overflows, the original order is untouched.
auto [targetLevelIt, createdLevel] = levels.try_emplace(orderEvent.price);
const std::optional<OrderHandle> newOrderHandle =
    targetLevelIt->second.addOrder(orderEvent.order_id, orderEvent.quantity);
if (!newOrderHandle.has_value()) {
    if (createdLevel) levels.erase(targetLevelIt);
    return failure(ApplyError::level_quantity_overflow);
}

ApplyOutcome outcome{.createdLevel = createdLevel};
oldLevelIt->second.removeOrder(locator.order_pos);
if (oldLevelIt->second.isEmpty()) {
    levels.erase(oldLevelIt);
    outcome.removedLevel = true;
}

locator.price = orderEvent.price; // locator is a reference into the index
locator.order_pos = *newOrderHandle;
return success(outcome);
```

✗ remove-then-insert (NOT used)



The order has already left the book when the failure is discovered. Restoring it would also lose its original queue position.

✓ insert-then-remove (used)

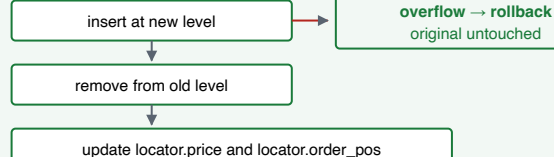


Figure 17.1 — The ordering is a correctness requirement, not a style choice: only the right-hand version can fail safely.

A price change **loses queue position** — the order joins the back of the destination level. That is correct market behaviour: a reprice is a new place in line.

`locator` is a non-const reference obtained from `orderIndex_.find()`, so the two assignments at the end update the index entry in place. No re-lookup, no re-insertion.

17.3 Failure modes

Error	Cause	State after
invalid_quantity	quantity ≤ 0	unchanged
invalid_price	price ≤ 0	unchanged
side_mismatch	Event's side ≠ stored locator's side	unchanged
unknown_order_id	Index names a level that does not exist	unchanged — indicates internal corruption, not bad input
level_quantity_overflow	Destination aggregate would exceed INT64_MAX	unchanged (rolled back)

TWO ERRORS, ONE NAME

`unknown_order_id` is returned both by `apply()` for a genuinely unseen ID (ordinary, expected on a feed gap) and by `applyModify` / `applyRemove` when the index points at a missing level (never ordinary — it means the two structures disagree). Callers cannot distinguish them. `Replay` 's redundant-removal tolerance (§22.4) keys on this same error, so a genuine index/level inconsistency on a remove could in principle be absorbed as a "redundant removal" rather than surfacing. A distinct `index_level_missing` variant would separate the two.

18 REMOVE lifecycle

`order_deleted` → `EventKind::remove` → the locator is authoritative

src/book/order_book.cpp:147–172 — `applyRemove`, complete

```
auto orderIt = orderIndex_.find(orderEvent.order_id);
assert(orderIt != orderIndex_.end());           // apply() already proved this
const OrderLocator& locator = orderIt->second;
if (orderEvent.side != locator.side) return failure(ApplyError::side_mismatch);

// The stored locator is authoritative; a delete message's price and quantity are not.
auto& levels = (locator.side == Side::buy) ? bids_ : asks_;
auto levelIt = levels.find(locator.price);
if (levelIt == levels.end()) return failure(ApplyError::unknown_order_id);

levelIt->second.removeOrder(locator.order_pos);

ApplyOutcome outcome{};
if (levelIt->second.isEmpty()) {
    levels.erase(levelIt);
    outcome.removedLevel = true;
}

orderIndex_.erase(orderIt);
return success(outcome);
```

18.1 Why the incoming price is ignored

The delete frame carries `price_str` and `amount_str`. The code uses neither. It uses `locator.price` — what the book recorded when the order was added or last repriced.

If the code trusted the message	Consequence
Message price ≠ actual resting price	Level lookup misses → order never removed → phantom liquidity forever
Message price matches a <i>different</i> level	Removes the wrong order, corrupting two levels at once
Message quantity is stale	Aggregate decremented by the wrong amount
Using the locator: the book removes exactly the node it inserted, decrementing by exactly the quantity it recorded. The message contributes only an ID and a side.	

The real Bitstamp delete frame in §7.1 shows why this matters: it carries `"amount": 0.00171371` and `"amount_traded": "0"`, values describing the order's history rather than necessarily its current resting state. The book's own record is the only trustworthy source for its own contents.

THE GENERAL PRINCIPLE

Never let a message describe state the receiver already owns. The sender's view is stale by definition — it was true when sent, not when processed. This is the same reasoning behind idempotency keys in payment APIs (trust your record of what you processed, not the retry's claim) and behind CRDTs resolving by identity rather than by transmitted value.

18.2 Removal order matters

`orderIndex_.erase(orderIt)` runs **last**, after the list node is gone. The reverse order would destroy `locator` — which is a reference into the index entry — while `locator.order_pos` is still needed to find the node. That would be a use-after-free, and it would frequently appear to work.

18.3 Empty-level cleanup

When the last order at a price leaves, the level is erased rather than kept at zero quantity. The book stays **sparse**: `levelCount()` counts only levels with liquidity, and `bestBid()` cannot return a price with nothing behind it. `digest()` reinforces this defensively by skipping zero-quantity levels even if one existed, so an absent level and an empty level can never produce different fingerprints.

BEFORE — remove order B (the only order at 101.00)

```
BIDS
100.00 → [A: 2]
ASKS
101.00 → [B: 1] ← single order
```

→

AFTER — level erased, not zeroed

```
BIDS
100.00 → [A: 2]
ASKS
(empty — asks_.begin() == asks_.end())
```

`ApplyOutcome{ createdLevel=false, removedLevel=true } · orderIndex_ loses B ·`

`marketShape()` now returns `one_sided`, not `open` — a visible consequence for any future `DecisionGate`.

Figure 18.1 — Level erasure changes market shape, which is exactly the signal the designed decision gate would consume.

19 Order state machine

Every legal transition, and every rejection.

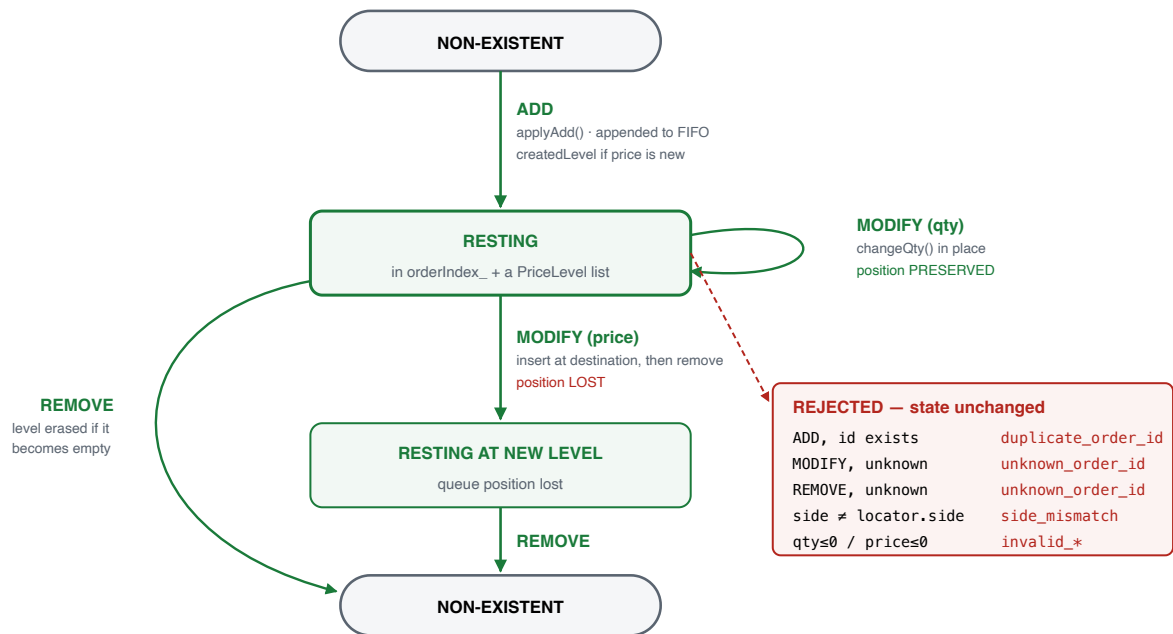


Figure 19.1 — Green = state change. Red = rejection with the book untouched. There is no partial-transition path.

19.1 Transition table

From	Event	To	Functions	Queue position
non-existent	add	resting	applyAdd → addOrder	back of FIFO
resting	modify (same price)	resting	changeQty	preserved
resting	modify (new price)	resting @ new level	addOrder → removeOrder	lost
resting	remove	non-existent	removeOrder → index erase	n/a
resting	add	— rejected	apply() gate	unchanged
non-existent	modify	— rejected	apply() gate	unchanged
non-existent	remove	— rejected	apply() gate	unchanged

19.2 Rejection is not always an error

`unknown_order_id` on a remove is the *expected* outcome in two legitimate situations: an order that existed before the seed snapshot, and an order a trade correction already removed. The first is handled by the replay window (§11.3); the second by tombstones (§22.4). The book itself takes no position on which — it reports the reason and lets the controller decide whether it means bad input or resynchronization.

`include/te/book/order_book.hpp:62`

```
// Every rejected event has a reason; callers decide whether it means bad input or resync.
[[nodiscard]] Result<ApplyOutcome, ApplyError> apply(const OrderEvent& orderEvent);
```

`[[nodiscard]]` makes ignoring the result a compiler warning. Given that a rejected event is normal traffic, silently dropping the return value would be an easy and invisible mistake.

20 Trades are not order events

Four distinct concepts that a careless design would merge into one.

Concept	Type	What it asserts
Order event	OrderEvent	"This order now rests at this price with this quantity." A statement about book state.
Trade event	TradeEvent	"These two order IDs exchanged this quantity." A statement about an execution.
Trade evidence	Qty amountTraded	"This order event already had this much subtracted." The venue's own admission, carried alongside the event.
Resting-book state	OrderBook	The engine's reconstruction. The only thing that is actually authoritative here.

include/te/feed/trade_event.hpp — complete

```
// One execution names both counterpart order IDs; either or both may have been resting.
struct TradeEvent {
    std::uint64_t venue_timestamp_us{};
    OrderId      buy_order_id{};
    OrderId      sell_order_id{};
    Qty          quantity{};
};
```

Four fields. No price, no side, no maker/taker flag. Each omission is deliberate:

- **No price** — reconciliation matches on ID and quantity; the resting price comes from the shadow record, which is more trustworthy than the trade's own report.
- **No side** — a trade has two sides by definition, and both IDs are checked independently.
- **No maker/taker flag** — the venue does not reliably say, and the reconciler does not need to know: it simply asks whether each ID is in its shadow of resting orders. The taker typically is not, which is normal and produces no correction.

20.1 A trade never applies directly to the book

This is the critical asymmetry. `OrderBook::apply()` accepts only `OrderEvent`. There is no `apply(TradeEvent)` overload and no `EventKind::trade`. A trade can influence the book **only** by causing the reconciler to emit a synthetic `OrderEvent`.

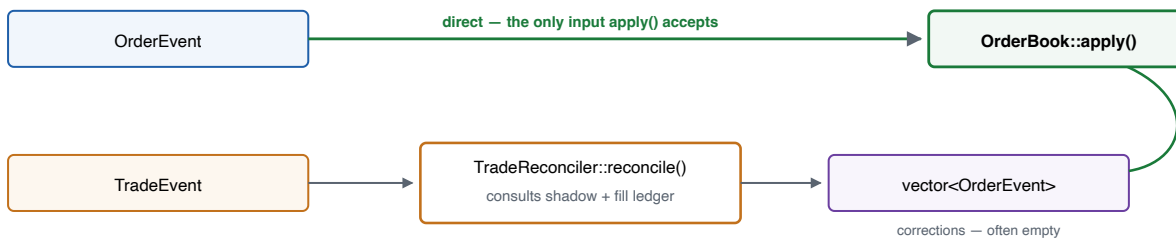


Figure 20.1 — Corrections re-enter through the same front door as real events, so they receive identical validation, identical rollback and identical digesting.

WHY CORRECTIONS REUSE THE EVENT TYPE

Representing a correction as an `OrderEvent` rather than a bespoke type means it cannot bypass validation, cannot skip the structural check, and appears in the applied-event digest exactly like a real event. Two runs that produce the same digest therefore agree on corrections too. This is the same instinct as event-sourcing systems that model compensating actions as ordinary events in the log rather than as out-of-band edits: the log stays the single complete description of how state was reached.

21 TradeReconciler in depth

The hardest correctness problem in the codebase, and the design that solved it.

21.1 The double-counting problem

Bitstamp reports a fill **twice**. The trade stream announces it, and `live_orders` *already subtracted it* from the resting quantity. ADR 0013's traced example, for order 2041946741522435 :

event	venue_ts	amount	amount_traded	amount_at_create
order_created	1787357128350000	0.10000000	0	0.10000000
TRADE	1787357128730000	0.05585426		
order_changed	1787357128730000	0.04414574	0.05585426	0.10000000
order_changed	1787357128731000	0.02489468	0.01925106	0.10000000
order_changed	1787357128731000	0.02250668	0.00238800	0.10000000
order_deleted	1787357128732000		0.02250668	0.10000000

At `...128730000` the trade and the `order_changed` share a timestamp. The order wins the tie, so it is processed first — and it already reports `0.04414574`, i.e. `0.10000000` minus the trade's `0.05585426`. A naïve reconciler then sees the trade, compares its `0.05585426` against the recorded `0.04414574`, concludes `trade ≥ resting`, and emits a **remove** for an order that should still hold `0.04414574`.

✗ NAÏVE — subtracts the same fill twice

book: order rests at `0.10000000`

1. `order_changed` → `0.04414574`

venue already subtracted `0.05585426`

`observe()` records `0.04414574`

2. trade of `0.05585426` arrives

`0.05585426 ≥ 0.04414574` ? YES

→ emit **REMOVE**

3. order GONE (should hold `0.04414574`)

4. next `order_changed` at `...731000`

→ `unknown_order_id` → **REPLAY ABORTS**

9 failures, 5 order IDs, real capture

ADR-recorded, before the fix

✓ FILL CREDIT — corrects only the shortfall

book: order rests at `0.10000000`

1. `order_changed` → `0.04414574`

`observe()` records `0.04414574`

+ CREDIT `0.05585426` @ `...730000`

2. trade of `0.05585426` @ `...730000`

credit timestamp matches →

`covered = 0.05585426`, `shortfall = 0`

3. `shortfall == 0` → NO correction

4. order still rests at `0.04414574` ✓

next `order_changed` applies cleanly

0 failures · 0 of 4,533 residuals

ADR-recorded — see §2.2 on reproducibility

Figure 21.1 — The fix is idempotence: apply only the part of a trade the venue has not already reported.

21.2 Why the credit must be keyed by timestamp

ADR 0013 records that the first design kept **one running credit per order**, with no timestamp — and that it is wrong. The counter-example is worth reproducing because the flaw is not obvious:

	book	credit	
start	10	0	
trade-only fill of 6 at T1	4	-6	(correction fires, correctly)
order_changed at T2, traded 1	3	-5	
matching trade of 1 at T2			credit is negative, so a correction fires AGAIN

The correct book quantity is 3, but a debt left over from T1 contaminates an unrelated fill at T2. The positive direction fails symmetrically: a credit of `+6` whose trade never arrives silently swallows a later genuine fill of 1.

The fix is to scope each credit to the key `(orderId, venueTimestampMicros)`. Credits at different timestamps are isolated and can never consume one another. Within a single timestamp the credit is a fungible pot — which is required, because ADR 0013 records that 10 orders in the reference capture carry more than one fill at the same microsecond.

21.3 The two data structures

include/te/feed/trade_reconciler.hpp:41-55

```
struct RestingInfo { Side side; Price price; Qty quantity; }; // shadow of the real book
struct AmountTradeInfo { std::uint64_t timestamp{}; Qty quantity{}; }; // a fill credit

std::unordered_map<OrderId, RestingInfo, OrderIdHash> resting_;
std::unordered_map<OrderId, AmountTradeInfo, OrderIdHash> fillLedge_;
```

Structure	Holds	Purpose
resting_	Side, price, quantity per order	A shadow of the real book. Lets <code>reconcile()</code> ask "was this ID resting, and how much?" without reading <code>OrderBook</code> .
fillLedge_	One (timestamp, quantity) credit per order	Records fills the venue already applied, so the matching trade can be recognised as covered.

The shadow exists so the reconciler **does not own or mutate** the real book — it returns corrections and lets the controller apply them. That keeps mutation authority in one place and makes the reconciler unit-testable without constructing a book at all.

21.4 observe() — maintaining the shadow

src/feed/trade_reconciler.cpp:6–56 (condensed)

```
case EventKind::add:
    resting_[event.order_id] = RestingInfo{event.side, event.price, event.quantity};
    break;

case EventKind::modify: {
    resting_[event.order_id] = RestingInfo{...}; // quantity is a level, not a delta
    auto ledgerIt = fillLedge_.find(event.order_id);
    if (ledgerIt == fillLedge_.end()) {
        if (amountTraded.units > 0)
            fillLedge_.emplace(event.order_id, AmountTradeInfo{event.venue_timestamp_us, amountTraded});
        break;
    }
    AmountTradeInfo& info = ledgerIt->second;
    if (info.timestamp == event.venue_timestamp_us) {
        info.quantity.units += amountTraded.units; // same microsecond: accumulate into the pot
    } else {
        if (info.quantity.units > 0) ++stats_.staleFillsDiscarded;
        if (amountTraded.units > 0) info = AmountTradeInfo{event.venue_timestamp_us, amountTraded};
        else fillLedge_.erase(ledgerIt);
    }
    break;
}
```

Because inputs are time-ordered, a credit from an older timestamp can no longer be matched — its trade is never coming. It is discarded and counted.

21.5 reconcile() — emitting corrections

src/feed/trade_reconciler.cpp:58–120 (condensed)

```
for (OrderId id : {trade.buy_order_id, trade.sell_order_id}) { // BOTH sides, independently
    const auto it = resting_.find(id);
    if (it == resting_.end()) continue; // normal: the taker never rested

    std::int64_t shortfallUnits = trade.quantity.units;

    auto ledgerIt = fillLedge_.find(id);
    if (ledgerIt != fillLedge_.end()) {
        AmountTradeInfo& credit = ledgerIt->second;
        if (credit.timestamp == trade.venue_timestamp_us) {
            const std::int64_t coveredUnits = std::min(shortfallUnits, credit.quantity.units);
            shortfallUnits -= coveredUnits;
            credit.quantity.units -= coveredUnits;
            if (credit.quantity.units == 0) fillLedge_.erase(ledgerIt);
        } else { ... ++stats_.staleFillsDiscarded; fillLedge_.erase(ledgerIt); }
    }

    if (shortfallUnits == 0) continue; // fully covered – no correction

    if (shortfallUnits >= restingInfo.quantity.units) {
        corrections.push_back(OrderEvent{ ..., .kind = EventKind::remove }); // FULL
        resting_.erase(it);
    } else {
        const Qty remaining{restingInfo.quantity.units - shortfallUnits};
        corrections.push_back(OrderEvent{ ..., .quantity = remaining, .kind = EventKind::modify }); // PARTIAL
        it->second.quantity = remaining;
    }
}
```

Condition	Correction	Shadow update
Shortfall = 0 (fully covered)	none	credit consumed
Shortfall ≥ resting quantity	<code>EventKind::remove</code>	erased from <code>resting_</code>
0 < shortfall < resting quantity	<code>EventKind::modify</code> to remainder	quantity reduced
ID not in shadow	none	unchanged — the taker case

21.6 Health counters that had to be redefined

ADR 0013 records that the first counter, `ordersRemovedWithOpenFillBalance`, read **21** on a capture with zero residuals — so it was not measuring a fault. All 21 were orders whose fill-carrying `order_changed` shared a timestamp with its `order_deleted`. Because order events win exact ties, the delete always erases the credit before the matching trade can clear it. That is *structural*, not a defect.

It was renamed `ordersRemovedWithUnmatchedFill` and restricted to credits from an **earlier** timestamp. It now reads 0, alongside `staleFillsDiscarded` at 0, and both are meaningful again: non-zero means one stream reported a fill the other never confirmed. The code carries that reasoning inline at `trade_reconciler.cpp:44–46` and `trade_reconciler.hpp:17–19`.

A METRIC THAT CANNOT FIRE IS NOT A METRIC

This is a good example of a broader discipline: a health counter whose baseline is a nonzero "normal" value teaches operators to ignore it. Redefining it so that **zero means healthy and any nonzero value demands investigation** is what makes it alertable. The same reasoning drives error-budget design in SRE practice.

21.7 The uncomfortable truth about this component

THE CORRECTION PATH HAS NEVER FIRED ON REAL DATA

ADR-recorded, across 1,059 seconds and 753 trades in three segments:

Segment	Trades	Naming a resting order	Covered by a credit	Uncovered
20260822T000512Z s0	84	69	69	0
20260827T235604Z s0	582	530	530	0
20260827T235604Z s1	87	38	38	0
total	753	637	637	0

637 of 637. Bitstamp's `live_orders` reported every single fill against a resting order. The reconciler is therefore **insurance, not a load-bearing path** on this venue as observed. It is correct, cheap and unit-tested — but it has never been validated against live venue data, because it has never fired against any.

ADR 0013 draws two conclusions that remain the right ones: stop capturing in the hope of finding the case (three captures produced zero reachable instances), and expect the real evidence to come from a venue whose feed does *not* report post-fill quantities.

It also weakens an earlier claim. The three hardcoded adjustments in `test_golden_replay.cpp` were once explained as orders "fully filled, where `live_orders` emits no delete." If `live_orders` reports every fill, that explanation is unlikely — and since that capture is order-only, the hypothesis cannot be tested against it. Those three remain **unexplained**, and this document does not describe them as proven silent fills.

22 The full replay loop

Replay::replay() from entry to *ReplayResult*.

22.1 Sequence

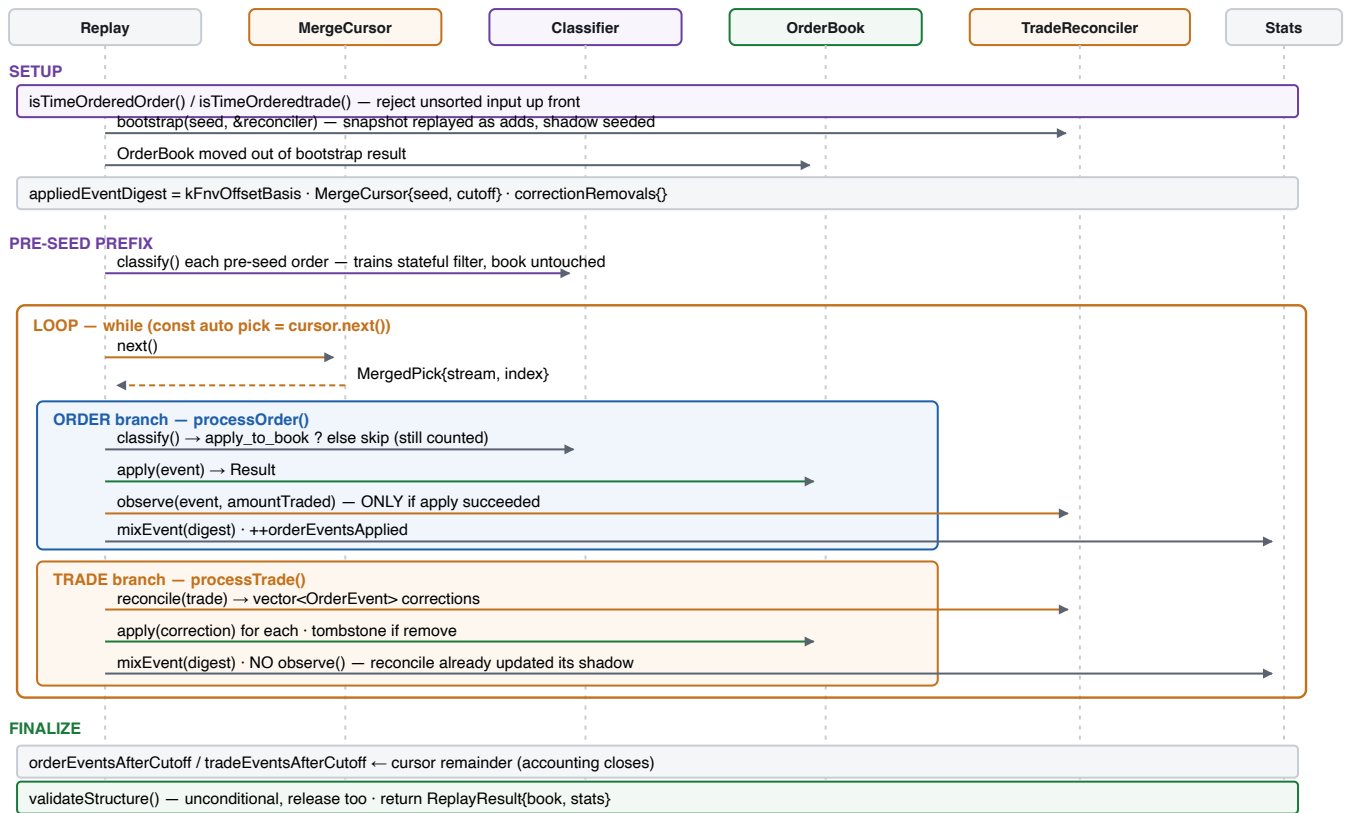


Figure 22.1 — The full loop. Note the asymmetry: order events are observed, generated corrections are not.

22.2 Order branch

src/feed/bitstamp/replay.cpp:53–85 — processOrder (condensed)

```

++replayStats.orderEventsRead;
if (classifier.classify(order.event) != EventDisposition::apply_to_book) return true; // skipped, counted

const auto result = orderBook.apply(order.event);
if (!result.hasValue()) { /* redundant-removal tolerance — see §22.4 */ }

if (order.event.kind == EventKind::add)
    correctionRemovals.erase(order.event.order_id); // ID reuse invalidates an old tombstone

// Observe only successfully applied raw events so the shadow matches the real book.
tradeReconciler.observe(order.event, order.amountTraded);
replayStats.appliedEventDigest = mixEvent(replayStats.appliedEventDigest, order.event);
++replayStats.orderEventsApplied;
  
```

`orderEventsRead` increments *before* classification, so filtered events still count as read. `orderEventsApplied` increments only on success. The gap between the two is exactly the classifier's filtering plus tolerated redundant removals.

22.3 Trade branch, and the asymmetry that matters

src/feed/bitstamp/replay.cpp:87–107 — processTrade (condensed)

```

const auto corrections = tradeReconciler.reconcile(trade.event);
for (const OrderEvent& correction : corrections) {
    const auto result = orderBook.apply(correction);
    if (!result.hasValue()) return false; // a correction MUST apply – no tolerance
    if (correction.kind == EventKind::remove)
        correctionRemovals.insert_or_assign(correction.order_id, correction.side);
    // reconcile() already updates its shadow; observing this correction would apply it twice.
    replayStats.appliedEventDigest = mixEvent(replayStats.appliedEventDigest, correction);
}

```

WHY CORRECTIONS ARE NEVER OBSERVED

`reconcile()` already mutated `resting_` when it decided to emit the correction. Calling `observe()` on the result would apply the same change a second time — precisely the double-counting class of bug this whole component exists to prevent, reintroduced one level up. The single-line comment in the source is the only thing standing between a future maintainer and that mistake.

Note also the asymmetry in failure handling: a failed *raw* event may be tolerated, but a failed *correction* aborts the replay immediately. A correction was generated from the engine's own shadow state, so if it cannot apply, the shadow and the book have diverged — an internal inconsistency, not a feed artifact.

22.4 The redundant-removal tolerance

src/feed/bitstamp/replay.cpp:63–75

```

// Tolerate only a later raw remove for an order this replay can prove a trade correction
// already removed. Other unknown removes and every failed modify remain real errors.
const auto correction = correctionRemovals.find(order.event.order_id);
if (order.event.kind == EventKind::remove && result.errorIf() != nullptr &&
    *result.errorIf() == ApplyError::unknown_order_id &&
    correction != correctionRemovals.end() && correction->second == order.event.side) {
    correctionRemovals.erase(correction);
    ++replayStats.redundantOrderRemovals;
    return true;
}
return false;

```

Four conditions must all hold: the event is a remove, it failed, it failed specifically with `unknown_order_id`, and this replay itself previously removed that ID on that side via a correction. The tombstone is then consumed — one tolerance per correction, not a standing permission.

THE NARROWNESS IS THE POINT

ADR 0013 explicitly rejects widening this to `modify`: that would "hide the symptom while leaving the book quantity wrong." A tolerance that absorbs a whole error class is indistinguishable from not checking. This one absorbs exactly the case the engine can *prove* it caused. Its only weakness is the shared `unknown_order_id` spelling noted in §17.3.

22.5 Replay error taxonomy

ReplayError	Raised when	Interpretation
order_input_not_time_ordered	Pre-flight scan fails	Caller bug or corrupt capture; the two-pointer merge is only valid on sorted input
trade_input_not_time_ordered	Pre-flight scan fails	Same
bootstrap_failure	A snapshot order fails to apply	Duplicate ID or invalid price/quantity in the seed
unexpected_order_apply_failure	A raw event fails and is not a tolerated redundant removal	The historically important one — this is what ADR 0013's nine double-counted removes produced
unexpected_correction_apply_failure	A generated correction fails to apply	Shadow and book have diverged — internal inconsistency

The two `isTimeOrdered*` pre-flight scans are $O(n)$ passes over the whole input before any work begins. They exist because the merge's correctness depends on an assumption the caller supplies, and an unchecked assumption would surface as a subtly wrong book rather than an error.

NAMING INCONSISTENCY

`isTimeOrderedOrder` / `isTimeOrderedtrade` — the second has a lowercase `t`. Cosmetic, but in an anonymous namespace two functions that should look like a matched pair do not.

23 Errors & defensive checks

Where each failure is detected, and whether state has already moved.

23.1 The error-handling philosophy

ADR 0002 forbids exceptions on these paths; ADR 0003 defines `Result<T, E>` as the carrier. The one exception handled anywhere in the book is `std::bad_alloc` around `orderIndex_.emplace`, and it is converted immediately into `ApplyError::allocation_failure`.

include/te/core/result.hpp – complete

```
template <typename T, typename E>
class Result {
public:
    static Result success(T value) { return Result(std::move(value)); }
    static Result failure(E error) { return Result(std::move(error)); }

    bool hasValue() const { return std::holds_alternative<T>(storage_); }

    // Pointer access makes querying the inactive alternative non-throwing; check before use.
    const T* valueIf() const { return std::get_if<T>(&storage_); }
    T* valueIf() { return std::get_if<T>(&storage_); }
    const E* errorIf() const { return std::get_if<E>(&storage_); }

private:
    std::variant<T, E> storage_;
    // Private constructors force call sites to state success or failure explicitly.
    explicit Result(T value) : storage_(std::move(value)) {}
    explicit Result(E error) : storage_(std::move(error)) {}
};
```

Private constructors plus named factories mean a call site cannot accidentally construct a success from an error-shaped value; it must say which it means. The constraint that `T` and `E` differ follows from querying the variant by type.

23.2 Complete error-path table

Failure	Detected in	State changed?	Rollback?	Continue or abort
malformed_json	decoder	no	n/a	Segment load aborts
missing_field / invalid_field	decoder	no	n/a	Segment load aborts
id_mismatch	decoder	no	n/a	Segment load aborts
not_order_event	decoder	no	n/a	Normal — control frames
overflow / excess_precision	parseDecimal	no	n/a	Becomes <code>invalid_field</code>
duplicate_order_id (snapshot)	parseSnapshot	no	n/a	Whole snapshot rejected
invalid_side / invalid_event_kind	apply() gate	no	n/a	Event rejected
duplicate_order_id	apply() gate	no	n/a	Event rejected
unknown_order_id	apply() gate	no	n/a	Tolerated only as redundant removal
invalid_price / invalid_quantity	applyAdd / applyModify	no	n/a	Event rejected
side_mismatch	applyModify / applyRemove	no	n/a	Event rejected
level_quantity_overflow	PriceLevel	yes — level may exist	yes	Event rejected, book intact
allocation_failure	orderIndex_.emplace	yes — order inserted	yes	Event rejected, book intact
frame_index_ended_early payload_ended_early	loadSegment	no	n/a	Capture rejected
unknown_stream_kind	loadSegment	no	n/a	Capture rejected
unexpected_order_apply_failure	processOrder	yes — partial replay	no	Replay aborts
unexpected_correction_apply_failure	processTrade	yes — partial replay	no	Replay aborts

Only two rows show state changed *with* rollback — the two `applyAdd` / `applyModify` paths analysed in §16–17. The replay-level aborts do leave a partially-replayed book, but it is discarded with the failed `Result` rather than returned.

23.3 validateStructure() — the internal oracle

src/book/order_book.cpp:196–228 (condensed)

```
// Every index entry names an existing level and points to the order with its key's ID.
for (const auto& [orderId, locator] : orderIndex_) {
    assert(levels.find(locator.price) != levels.end());
    assert(locator.order_pos->id == orderId);
}

// Every order points back to this exact side, price, and list node, and totals are exact.
for (const auto& [price, level] : levels) {
    assert(!level.isEmpty()); // no empty levels may linger
    Qty currentQty{0};
    for (auto orderIt = level.begin(); orderIt != level.end(); ++orderIt) {
        const auto indexIt = orderIndex_.find(orderIt->id);
        assert(indexIt != orderIndex_.end());
        assert(locator.side == expectedSide);
        assert(locator.price == price);
        assert(locator.order_pos == orderIt); // exact node identity, not equality
        currentQty.units += orderIt->qty.units;
    }
    assert(level.totalQuantity() == currentQty); // cached aggregate is exact
}
```

The check is **bidirectional**: index→level→node, and node→index. A one-directional check would miss an orphaned list node that no index entry names — invisible to lookups, but it would still contribute to the cached total and therefore to `digest()`.

`assert(locator.order_pos == orderIt)` compares *iterators*, not values. Two orders with identical ID and quantity at the same price would compare equal by value but must still be distinct nodes.

Build	When it runs	Cost
Debug (NDEBUG unset)	After every successful <code>apply()</code>	$O(N+L)$ per event — expensive, deliberately
Release	Compiled out of <code>apply()</code> ; <code>Replay</code> still calls it once at the end	$O(N+L)$ once per replay

Note that `replay.cpp:172` calls `orderBook.validateStructure()` unconditionally before returning, and `bootstrap.cpp:28` does the same after seeding. In a release build those calls still execute — but the `assert` s inside them are compiled away, so the function walks the entire book and checks nothing. That is wasted $O(N+L)$ work rather than a correctness problem, and it means release replay does **not** in fact have the "explicit checkpoint" validation ADR 0014 describes.

FINDING — THE RELEASE CHECKPOINT IS A NO-OP

ADR 0014 states "Release replay may run it at declared checkpoints." Because `validateStructure()` is built entirely from `assert`, calling it in a release build costs a full traversal and verifies nothing. Making the checkpoint real would require checks that survive `NDEBUG` — returning a `Result` rather than asserting.

24 Determinism

The property everything else is arranged to protect.

24.1 The six mechanisms

Mechanism	Implementation	What it rules out
Exact integer arithmetic	<code>int64</code> ticks everywhere	Float drift; order-dependent rounding
Centralized tie-break	<code>MergeCursor::next()</code> alone	Two components disagreeing on sequence
Ordered input requirement	Pre-flight <code>isTimeOrdered*</code>	A merge that is silently invalid
Explicit replay window	<code>seed < ts ≤ cutoff + counters</code>	Ambiguity about which events participate
Semantic digests	FNV-1a over fields in canonical order	Layout, padding, allocator and compiler variation
Structural invariants	<code>validateStructure()</code>	Silent internal divergence

24.2 Two digests, two questions

Digest	Covers	Answers
<code>OrderBook::digest()</code>	Every non-empty level as (side, price, aggregate qty)	"Do two books hold the same liquidity at the same prices?"
<code>ReplayStats::appliedEventDigest</code>	Every applied event — raw and corrective — by all six fields	"Did two runs process the same tape the same way?"

Both are needed. Two runs could reach the same final book by different routes; the event digest catches that. Conversely, the book digest is the only one a **snapshot can independently verify**.

24.3 What `digest()` deliberately excludes

`include/te/book/order_book.hpp:90–94`

```
// Deliberately excludes order IDs and per-order queue position: this answers "is the
// same liquidity resting at the same prices", which is what a checkpoint snapshot can
// independently confirm. A snapshot cannot confirm queue order, so hashing it would
// produce a digest nothing external could ever verify.
```

This is a subtle and correct piece of reasoning. A Bitstamp REST snapshot lists price, amount and ID per level but **cannot** tell you the FIFO order of orders at the same price. Hashing queue position would produce a number no external source could ever corroborate — a self-referential check that only ever compares the system to itself.

The cost is real and worth stating: two books with identical aggregate liquidity but *different queue orders* produce the same digest. Since queue position is exactly what L3 exists to provide, a future queue-model component cannot rely on `digest()` to prove its ordering is right; it will need its own verification.

24.4 The canonical walk

`src/book/order_book.cpp:248–277` (condensed)

```
// FNV-1a. std::map iterates in ascending price order, so the walk is already canonical and
// does not depend on the order events arrived in.
std::uint64_t digest = kOffsetBasis;
for (const auto& [side, levels] : {std::pair{Side::buy, &bids_},
                                   std::pair{Side::sell, &asks_}}) {
    for (const auto& [price, level] : *levels) {
        // An empty level is indistinguishable from an absent one for qtyAt, so it must not
        // change the digest either.
        if (level.totalQuantity().units == 0) continue;
        digest = mix(digest, static_cast<std::uint64_t>(side));
        digest = mix(digest, static_cast<std::uint64_t>(price.ticks));
        digest = mix(digest, static_cast<std::uint64_t>(level.totalQuantity().units));
    }
}
```

`std::map` provides the canonical order for free — bids and asks are each walked in ascending price. Had the book used `unordered_map` for levels, the walk would have to sort first or the digest would vary by bucket layout.

The `mix` function feeds eight bytes individually rather than hashing the 64-bit word wholesale, which makes the result independent of host endianness. That matters for the stated goal of comparing books "across compilers."

WHY HASH SEMANTICS, NOT BYTES

`memcpy` -hashing a struct includes padding bytes, which are indeterminate — two logically identical objects can hash differently on the same machine. Field-by-field hashing in a defined order is the standard fix, and it is why formats like Protocol Buffers specify canonicalization rules and why Merkle-tree systems (Git, blockchains, Certificate Transparency) hash a defined serialization rather than in-memory representation.

24.5 The remaining determinism gap

ITERATION ORDER THAT IS STABLE BUT NOT SPECIFIED

`TradeReconciler` uses `std::unordered_map` for both `resting_` and `fillLedger_`. Neither is *iterated* — both are only looked up by key — so no ordering dependency exists today. Similarly `correctionRemovals` in `Replay` is lookup-only, and `zeroPriceOrders_` in the classifier is an `unordered_set` used only for membership. Determinism therefore holds. But it holds by *current usage*, not by construction: adding an iteration over any of these would introduce implementation-defined ordering into a system whose entire value proposition is reproducibility. Nothing in the code currently prevents that.

25 Tests as evidence

What the suite actually proves — and what it cannot.

25.1 Suite composition

Group	Count	Notes
Unit tests	~283	Fast, hermetic, no external data
Golden tests (label <code>golden</code>)	21	Byte-exact / end-to-end
Corpus tests (label <code>corpus</code>)	2	Require private capture data; designed to skip when absent
Total registered	304	303 pass, 1 fails (§2.2)

Total execution time: ~2.1 s wall clock for the full suite.

25.2 Behaviour → test → what it demonstrates

Behaviour	Production code	Test proving it	What it demonstrates
Exact decimal conversion + overflow rejection	<code>text_to_int.cpp</code>	<code>test_decimal.cpp</code> (238 lines)	Guarded arithmetic rejects rather than wraps
Field mapping, <code>id</code> / <code>id_str</code> agreement	<code>decoder.cpp</code>	<code>test_decoder.cpp</code>	Real captured lines decode to expected integers
Snapshot parsing, duplicate rejection	<code>snapshot.cpp</code>	<code>test_bitstamp_snapshot.cpp</code> (316)	Ambiguous seeds are refused whole
Tie-break, window boundaries, accounting	<code>merge_cursor.cpp</code>	<code>test_merge_cursor.cpp</code> (195)	ADR 0013's rule is pinned by test, not convention
Add/modify/remove, rollback, invariants	<code>order_book.cpp</code>	<code>test_order_book.cpp</code> (385)	Failure atomicity on every rejected mutation
Reference-book correctness	<code>order_book.cpp</code>	<code>test_book_oracle.cpp</code> (251)	Independent oracle comparison
Fill credits, shortfall, stale discard	<code>trade_reconciler.cpp</code>	<code>test_trade_reconciler.cpp</code> (157)	Correction logic — synthetic only (§25.4)
Health-state transitions	<code>book_health.cpp</code>	<code>test_book_health.cpp</code> (95)	Includes the resync reason-clearing fix in <code>d00e244</code>
End-to-end joined replay	<code>replay.cpp</code>	<code>test_bitstamp_joined_capture.cpp</code> (405)	Synthetic case passes; real-corpus case failing
Byte-exact record round-trip	<code>record.cpp</code>	<code>test_recorder.cpp</code> (321)	Padding zeroed; two builds produce identical bytes
v3 tape format	<code>segment_format.cpp</code>	<code>test_segment_format.cpp</code> (456)	Header validation, version and policy gating

25.3 The committed synthetic fixture

`test_bitstamp_joined_capture.cpp` builds a complete capture directory in a temp folder — manifest, seed, checkpoint, payload, frame index — from string constants, then runs the real loader and the real replay over it. The value is that it needs **no external data**, so the whole pipeline is protected on a fresh checkout. Its constants are real Bitstamp frames (§7.1, §7.4), not invented ones.

That fixture is also what makes §2.2's failure diagnosable: the synthetic cases pass while only the real-corpus case fails, which localises the problem to the data rather than the code.

25.4 What the suite does not prove

HONEST LIMITS

- **The correction path is unexercised by reality.** Every correction test is synthetic. ADR-recorded: 0 of 753 real trades produced an uncovered shortfall.
- **The real-corpus gate is currently red** and, because the data is hollow, cannot be restored by re-running.
- **BookHealth has no production caller.** Its transitions are unit-tested, but nothing in `Replay`, `loadSegment` or `captureCoordinator` ever constructs one. It is tested, working, and unused.
- **Three unexplained adjustments** remain in `test_golden_replay.cpp`. The original explanation is now doubtful (§21.7) and the capture cannot test the hypothesis.
- **No performance claims are tested.** There are no benchmarks in the suite; this document makes no latency or throughput claims.
- **Release-build validation is vacuous** (§23.3).

26 One full worked scenario

Nine steps, tracing every structure at every point.

Starting book, after bootstrap from a seed at $t = 1000$ (prices at scale 2, quantities at scale 8; displayed in human units):

```
BIDS          ASKS
100.00 → [A: 5] 101.00 → [C: 4]
99.00 → [B: 3]

orderIndex_: A→{buy,10000,.} B→{buy,9900,.} C→{sell,10100,.}
reconciler.resting_: A,B,C seeded with quantity, no fill credits
digest(): H0      marketShape(): open      levelCount(): 3
```

#	Input	ts	Function path	Book after	Reconciler after	Stats
1	ORDER add D, buy 2 @ 100.00	1100	next()→order classify→apply applyAdd	100.00 → [A:5, D:2] = 7 D appended behind A	resting_ _[D] = 2	read 1, applied 1
2	ORDER modify A → 4, amount_traded = 1	1200	applyModify Case A: changeQty	100.00 → [A:4, D:2] = 6 A keeps queue position	resting_ _[A] = 4 fillLedge _[A] = (1200, 1)	read 2, applied 2
3	TRADE A↔X, qty 1	1200	next()→trade reconcile()	unchanged — 100.00 → [A:4, D:2]	credit timestamp matches → covered 1, shortfall 0 credit erased	trades 1, corrections 0
Step 3 is the whole point of §21. The venue already subtracted this fill at step 2. Without the credit, the reconciler would compare trade qty 1 against resting 4, emit a <code>modify</code> to 3, and the book would be wrong by exactly one fill. X is the taker and is not in <code>resting_</code> , so it is skipped.						
4	ORDER modify D → 100.50, qty 2	1300	applyModify Case B: insert→remove	100.50 → [D:2] (<i>new</i>) 100.00 → [A:4] D loses position	resting_ _[D] price 100.50	applied 3, level created
5	TRADE B↔Y, qty 3	1400	reconcile()	99.00 level erased via generated remove	no credit for B → shortfall 3 ≥ resting 3 FULL correction ; resting_ _[B] loses B	corrections gen 1, applied 1 tombstone B→buy
6	ORDER remove B	1450	apply → unknown_order_id tombstone matched	unchanged	unchanged	redundantOrderRemovals 1 tombstone consumed
Step 6 is the tolerance in §22.4. All four conditions hold: remove, failed, <code>unknown_order_id</code> , matching tombstone with matching side. Replay continues instead of aborting. The tombstone is erased, so a second stray remove for B <i>would</i> abort.						
7	TRADE C↔Z, qty 1	1500	reconcile()	101.00 → [C:3] via generated modify	no credit → shortfall 1 < resting 4 PARTIAL correction ; resting_ _[C] =3	corrections gen 2, applied 2
8	ORDER remove C	1600	applyRemove	101.00 erased ASKS now empty	resting_ _[C] loses C	applied 4, level removed
9	Final state. BIDS: 100.50 → [D:2], 100.00 → [A:4]. ASKS: empty. levelCount() = 2. bestBid() = 100.50, bestAsk() = nullopt → marketShape() is now one_sided , not open. A future <code>DecisionGate</code> would block here with a named shape reason even though book trust is perfect — the §4 separation made concrete.					

26.1 Final accounting

```
orderEventsBeforeSeed 0
orderEventsRead       5   (steps 1,2,4,6,8)
orderEventsApplied    4   (step 6 tolerated, not applied)
orderEventsAfterCutoff 0
→ accountedFor = 0 + 5 + 0 = 5 = input size ✓ conservation holds

tradeEventsRead       3   (steps 3,5,7)
correctionsGenerated  2   (steps 5,7 – step 3 generated none)
correctionsApplied    2
redundantOrderRemovals 1   (step 6)
reconciler.ordersRemovedWithUnmatchedFill 0
reconciler.staleFillsDiscarded 0

appliedEventDigest = FNV-1a over 6 events: 4 raw + 2 corrections
digest()           = FNV-1a over 2 levels: (buy,10050,2) then (buy,10000,4)
```

Six events entered the applied-event digest, not four: generated corrections are digested exactly like real events (§20.1). The book digest walks bids in ascending price order — 10000 before 10050 — regardless of the order events arrived in.

27 Implemented vs. designed

The maturity map, stated without optimism.

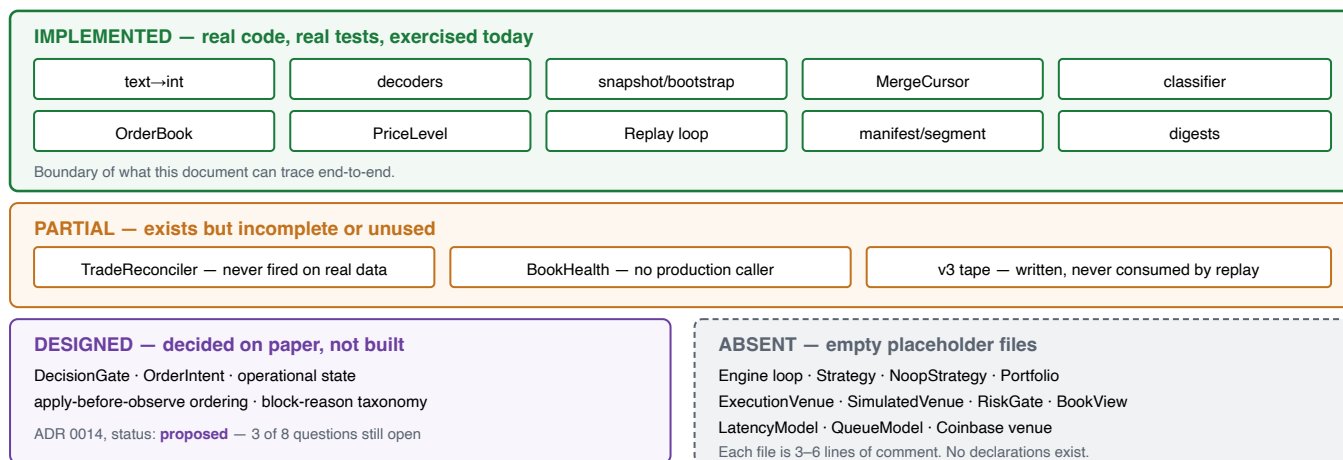


Figure 27.1 — Everything below the green band is either unused, unbuilt, or undecided.

27.1 Verbatim proof of the placeholders

These are complete file contents, not excerpts:

```
// ===== include/te/engine/engine.hpp =====
#pragma once
// Reserved for plan v4 §15 (Stage 5). Not implemented. Deliberately simple event loop and
// injected Feed/Clock/Strategy/ExecutionVenue wiring, test-first.
namespace te {} // namespace te

// ===== include/te/engine/portfolio.hpp =====
#pragma once
// Reserved for plan v4 §15 (Stage 5). Not implemented. Positions and PnL, test-first, using
// int64 ticks, never double.
namespace te {} // namespace te

// ===== strategies/noop_strategy.cpp =====
// Reserved for plan v4 §15 (Stage 5). Trades nothing. The gate: a full run through this strategy
// conserves event counts, cash, position and PnL exactly.
// TODO(fuaad): write this yourself.

// ===== tests/unit/test_portfolio.cpp =====
// Slice 3. Write the test BEFORE the implementation.
// TODO(fuaad): write this yourself.
```

apps/replay_main.cpp is a main() that returns 0 and is commented out of the CMake build. Nine placeholder headers and three placeholder strategy files follow the same pattern. The corresponding test files are empty too, which is consistent with the project's test-first discipline — neither side has been written.

27.2 Honest phrasing rules applied in this document

Never written here	Written instead
"The engine processes strategy decisions"	"No engine exists; ADR 0014 designs one"
"The risk gate rejects oversized orders"	" risk.hpp is an empty placeholder"
"Reconciliation corrects venue fills"	"Reconciliation <i>can</i> correct; it never has on observed data"
"The book validates after every event"	"...in debug builds; the release call verifies nothing"
"303 of 303 tests pass"	"304 registered; 303 pass, 1 fails — here is why"

28 The decision frontier

► **THIS IS WHERE THE CODE ENDS AND CURRENT DESIGN WORK BEGINS.**

HOW TO READ THIS CHAPTER

Everything in §1–§27 describes code that exists. **Nothing in this chapter is implemented.** ADR 0014 is *proposed*, not accepted, and it is the gate blocking all engine work. This chapter records what has been settled so far — including decisions taken in design sessions after the ADR was drafted — and what remains genuinely open.

28.1 Settled in ADR 0014 as drafted

Decision	Consequence
Apply before observe	An accepted event mutates the book <i>before</i> a strategy sees it. Strategies observe post-event state only.
Observation is unconditional	Every successfully processed event reaches the strategy — even when the market is not decision-ready. Filtering would make stateful indicators drift.
Observation ≠ authority	A strategy never gets a mutable book or the venue. It returns <code>OrderIntent</code> values; an intent is data, not an order.
Shape only at a safe checkpoint	Market shape is evaluated after all mutations from one logical input complete. Intermediate crossed states are not diagnosed.
One engine-owned gate	<code>DecisionGate</code> combines trust, shape and operational state. Strategies must not reproduce this policy.
Structural failure fails closed	A failed <code>validateStructure()</code> is an internal defect, not a market condition. Terminate with <code>internal_invariant_violation</code> ; resync is not a valid repair for inconsistent code.
Replay owns its control inputs	Operational transitions are scripted data in the event sequence. No wall-clock reads, no unscripted operator actions.

28.2 Resolved in design sessions after the draft

Five of the eight open questions have since been answered. These are **not yet written into the ADR** — the ADR text still lists all eight as open.

#	Question	Resolution and reasoning
1	Minimal Stage 5 operational states	Defer <code>OperationalState</code> entirely to Stage 9. At Stage 5 there is no operator, no kill switch, and a file cannot disconnect — so the type could only ever hold one value, and no test could distinguish it from absent. <code>DecisionGate</code> therefore combines <i>two</i> inputs at Stage 5: book trust and market shape. Same reasoning the project already applies to banning <code>return true</code> risk stubs: a check that cannot fail is not a check.
2	When an intent enters the latency queue	Only after both <code>DecisionGate</code> and the risk check approve it. The latency queue models <i>wire time to the venue</i> , not deliberation time, so nothing may occupy it before the engine has decided to send. A rejected intent never touches the queue at all.
3	First fill eligibility	At the arrival instant , against the book as it stands at that exact moment — not the book as it was when the strategy decided. The stricter alternative (wait for subsequent market activity) was considered and judged not worth the added complexity at this stage.
4	Deterministic timestamp ordering	Three rules. (a) Venue timestamp is the only ordering clock; receipt timestamp is ignored, because it varies with local network conditions and would make identical input replay differently — the standard <i>event time</i> vs. <i>processing time</i> distinction from stream processing. (b) On a tie between a real market event and a synthetic order arrival, the real event applies first , so a synthetic order can never jump ahead of history. (c) On a tie between two of the engine's own orders, the lower submission sequence number wins — a monotonic per-intent counter, mirroring the FIFO time priority <code>PriceLevel</code> already implements.
5	Fill / fee / cash / position / PnL sequence	Venue confirms the fill → compute fee → on a buy , fold price and fee into a new weighted average entry price; on a sell , use the <i>existing</i> average price to compute realized PnL on the quantity sold, with the fee reducing that PnL → then update cash, position, average price and PnL atomically , so nothing observes a half-updated portfolio. A partial sell does not change the average price of the remainder, and realizes PnL only on the quantity actually sold. This mirrors standard cost-basis accounting: acquisition costs raise basis, disposal costs reduce proceeds. The atomicity requirement matches the failure atomicity <code>OrderBook</code> already guarantees.

28.3 Still open

#	Question	Current state
6	Observation and decision callbacks while untrusted	Observation is settled (unconditional). What is open: whether <code>DecisionGate</code> is still <i>invoked</i> during untrusted states to produce a named block reason, or skipped as a fast path. The evidence gate requiring "every blocked decision counted by its named reason" argues for always invoking it. Note <code>disconnected</code> / <code>gapped</code> / <code>resynchronizing</code> are out of scope at Stage 5 per question 1.
7	Which reason taxonomies are permanent contracts	Open. The narrow option — only reasons a Stage 5 test asserts by name become permanent — matches the project's established "don't lock in more than is load-bearing" instinct. The broad option mirrors exchange practice, where a published reject code is never removed, only added to.
8	Crossed-checkpoint → lost trust threshold	Open, and deliberately un-guessable : the TODO explicitly forbids inventing a threshold without evidence. The viable path is to block on <code>crossed</code> every time via market shape, log crossed-checkpoint durations in the replay report, and set a threshold once the corpus shows how long real transient crosses last. Compare flap-damping in BGP or liveness failure thresholds in Kubernetes — the window is measured, not assumed.

28.4 The timeline as currently settled

T0	input market event selected by deterministic replay ordering	[settled]
T1	decode and classify	[settled · implemented today]
T2	apply / reconcile all mutations from that logical input	[settled · implemented today]
T3	debug structural validation	[settled · implemented today]
T4	safe checkpoint: compute trust + market shape	[settled – no operational state at Stage 5]
T5	strategy observes post-event BookView, unconditionally	[settled]
T6	DecisionGate: allow, or record a named block reason	[partly open – Q6]
T7	strategy returns zero or more OrderIntent values	[settled]
T8	risk/admission accepts or rejects each intent by reason	[taxonomy open – Q7]
T9	accepted intent enters the outbound latency queue	[settled – only after T6 and T8 pass]
T10	arrival at simulated venue; eligible to fill at this instant	[settled]
	· real market events at the same timestamp apply first	
	· own orders tie-break by submission sequence number	
T11	fill → fee → average price or realized PnL → atomic commit	[settled]

28.5 What must happen before any engine code is written

1. Answer questions 6, 7 and 8.
2. Fold the §28.2 resolutions into ADR 0014's text — they currently live only in session history, which is exactly the drift this project's documentation rules exist to prevent.
3. Move ADR 0014 from **proposed** to **accepted**, including at least one hand-worked timeline and a deterministic reason for every blocked decision and rejected order.
4. Only then start `T0D0.md` item 2 (exact-integer portfolio, test-first).

SEQUENCING IS LOAD-BEARING

Items 2–8 of `T0D0.md` all consume contracts this ADR defines. Building the portfolio before question 5 was settled would have meant discovering the fee/average-price rule mid-implementation — which is precisely the failure mode ADR 0013 documents from the merge design, where a first-principles answer turned out to be wrong and cost a rewrite.

29 Code-to-runtime cross reference

Every stage of the implemented path, in execution order.

Runtime stage	Source file	Function	Called by	Calls	Reads	Mutates
Read manifest	capture/manifest_reader.cpp	manifestReader()	captureCoordinator	simdjson	manifest.json	—
Load segment	capture/segment_loader.cpp	loadSegment()	captureCoordinator	parseSnapshot, decodeCapturedOrder, decodeTrade	payload, frames, snapshots	JoinedCapture
Parse snapshot	feed/bitstamp/snapshot.cpp	parseSnapshot()	loadSegment	parseDecimal, parseInteger	snapshot text	BookSnapshot
Decode order	feed/bitstamp/decoder.cpp	decodeCapturedOrder()	loadSegment	parseInteger, parseDecimal	JSON line	—
Decode trade	feed/bitstamp/trade_decoder.cpp	decodeTrade()	loadSegment	parseInteger, parseDecimal	JSON line	—
Convert text	core/text_to_int.cpp	parseDecimal() / parseInteger()	all decoders	—	string_view	—
Bootstrap	feed/bitstamp/bootstrap.cpp	bootstrap()	Replay::replay	apply, observe, validateStructure	BookSnapshot	OrderBook, resting_
Validate input order	feed/bitstamp/replay.cpp	isTimeOrderedOrder/trade()	Replay::replay	—	both vectors	—
Merge	feed/merge_cursor.cpp	MergeCursor::next()	Replay::replay	—	both spans	orderPtr_, tradePtr_
Classify	feed/bitstamp/classifier.cpp	classify()	processOrder	—	OrderEvent	zeroPriceOrders_, stats_
Apply (dispatch)	book/order_book.cpp	apply()	processOrder, processTrade, bootstrap	applyAdd/Modify/Remove, validateStructure	orderIndex_	dispatches
Apply add	book/order_book.cpp	applyAdd()	apply	try_emplace, addOrder	OrderEvent	bids_/asks_, orderIndex_
Apply modify	book/order_book.cpp	applyModify()	apply	changeQty, addOrder, removeOrder	orderIndex_	levels, locator
Apply remove	book/order_book.cpp	applyRemove()	apply	removeOrder	orderIndex_	levels, orderIndex_
Level ops	book/price_level.cpp	addOrder/removeOrder/changeQty	applyAdd/Modify/Remove	std::list	total_quantity_	restingOrders_, total_quantity_
Observe fill	feed/trade_reconciler.cpp	observe()	processOrder, bootstrap	—	OrderEvent, amountTraded	resting_, fillLedger_, stats_
Reconcile trade	feed/trade_reconciler.cpp	reconcile()	processTrade	—	TradeEvent	resting_, fillLedger_, stats_
Structural check	book/order_book.cpp	validateStructure()	apply (debug), Replay, bootstrap	—	all three containers	— (asserts only)
Book digest	book/order_book.cpp	digest()	captureCoordinator	—	bids_, asks_	—
Event digest	feed/bitstamp/replay.cpp	mixEvent()	processOrder, processTrade	mixDigest	OrderEvent	appliedEventDigest
Market shape	book/order_book.cpp	marketShape()	<i>no caller yet</i>	bestBid, bestAsk	bids_, asks_	—
Health state	book/book_health.cpp	startSynchronization() etc.	<i>no caller yet</i>	—	state_	state_, failureReason_
Checkpoint compare	capture/capture_coordinator.cpp	captureCoordinator()	tests	manifestReader, loadSegment, replay, qtyAt, digest	capture dir	CaptureReplayReport

Two rows have *no caller yet*: `marketShape()` and all of `BookHealth`. Both are built and tested in anticipation of the `DecisionGate` designed in §28.

30 Glossary

Repository-specific terminology.

Term	Definition
L2	Aggregated book: price levels with total quantity. Cannot identify individual orders.
L3	Order-by-order book: every order individually identified. What <code>live_orders</code> provides and what makes queue position knowable.
Resting order	An order in the book awaiting execution. Stored as <code>RestingOrder{id, qty}</code> in a <code>PriceLevel</code> list.
Order lifecycle	create → zero or more changes → delete. The classifier tracks whole lifecycles, not individual messages.
Snapshot	Complete book state at one instant, from REST. Has a <code>microtimestamp</code> .
Seed (S0)	The snapshot replay starts from. Its timestamp is the exclusive lower bound of the replay window.
Checkpoint (S1)	A second, independently fetched snapshot used to verify the replayed book. Its timestamp is normally the cutoff.
Cutoff	Inclusive upper bound of the replay window.
Captured event	A normalized event plus capture metadata (<code>amountTraded</code> , <code>captureOrdinal</code>).
Venue timestamp	<code>microtimestamp</code> from the exchange. The only clock used for ordering.
Capture ordinal	Local arrival sequence number. Carried as provenance; deliberately not the tie-break (§14.3).
Order stream / trade stream	The two independently time-ordered inputs that <code>MergeCursor</code> interleaves.
Reconciliation	Comparing the trade stream against the reconstructed book and emitting corrections for genuine disagreements.
Fill credit	A recorded <code>amount_traded</code> the venue already subtracted, keyed (<code>orderId</code> , <code>venueTimestampMicros</code>).
Shortfall	Trade quantity minus the matching fill credit. Only the shortfall produces a correction.
Correction event	A synthetic <code>OrderEvent</code> (modify or remove) generated by the reconciler.
Tombstone	An entry in <code>correctionRemovals</code> recording that a correction removed an ID, so one later raw remove is tolerated.
Chain break	A discontinuity in <code>event_id</code> / <code>pre_event_id</code> . Ends the capture segment; never replayed across.
Deterministic replay	Identical inputs produce identical book, digests and statistics on every run, on any machine.
Book digest	FNV-1a over (side, price, aggregate qty) for every non-empty level, in ascending price order.
Applied-event digest	FNV-1a over all six fields of every applied event, raw and corrective.
Price level	All orders at one price, in FIFO order, plus a cached exact aggregate.
Order locator	<code>{side, price, list iterator}</code> — the index's pointer to an order's exact node. Authoritative over any message's claims.
Bootstrap	Converting a snapshot into a book by replaying each entry as a synthetic <code>add</code> .
Book trust	Whether the reconstruction rests on a valid snapshot and an unbroken stream. <code>BookHealth::isTrusted()</code> .
Market shape	Visible best-bid/best-ask relationship: empty, one-sided, locked, crossed, open.
Decision readiness	The future combination of trust, shape, operational state and risk policy. Not implemented.

§ One-page cheat sheet

Print this page and keep it beside the keyboard.

